

DEEP LEARNING

Module-II

SYLLABUS

- **Module I: Introduction to Neural Networks**
- **Module II: Improving Neural Networks**
- **Module III: Convolutional Neural Networks (CNNs)**
- **Module IV: Recurrent Neural Networks (RNNs) and LSTMs**
- **Module V: Applications of Deep Learning**

Model II: Improving Neural Networks

- Hyperparameter Tuning
- Overfitting and Underfitting
- Regularization and Optimization
- Dropout and Batch Normalization
- Data Augmentation

Hyperparameter Tuning

- Hyperparameter tuning is the **process** of **selecting** the **optimal values** for a **deep learning model's** hyperparameters.
- These are **typically set before the actual training process begins** and control aspects of the learning process itself. They influence the model's performance its complexity and how fast it learns.
- These settings can **affect both** the **speed** and **quality** of the **model's performance**.

Hyperparameter Tuning

- For example, A **high learning rate** can **cause** the model to **converge too quickly possibly skipping** over the **optimal solution**.
- A **low learning rate** might lead to slower convergence and **require more time** and **computational resources**.
- **Different models** have **different hyperparameters** and they **need** to be **tuned accordingly**.

Hyperparameter Tuning Real-time Examples

- **PARAMETERS** = What the model **LEARNS** from data (internal, automatic)
- **HYPERPARAMETERS** = What **YOU** decide before training starts (external, manual)
- **Example** in baking a cake: **hyperparameters** are the **recipe decisions** you **make** beforehand (oven temperature, baking time, which pan size) — **parameters** are **what naturally emerges inside** the **cake** as it **bakes** (the **exact crumb structure, moisture distribution**) as a result of those recipe choices.

Hyperparameter Tuning Real-time Examples

- **Features** = The **INPUT DATA** columns we feed into the model
- **Parameters** = What the model **LEARNS** from data (internal, automatic)
- **Hyperparameters** = What **WE** decide before training starts (external, manual)
- **Example** in **baking** a **cake**: **hyperparameters** are the **recipe decisions** you make beforehand (oven temperature, baking time, which pan size) — **parameters** are **what naturally emerges inside** the **cake** as it **bakes** (the **exact crumb structure, moisture distribution**) as a result of those recipe choices.

Hyperparameter Tuning Real-time Examples

Example 1: Housing Price Prediction

Imagine predicting house price from square footage, number of bedrooms, and location score, using a simple linear regression:

$$\text{Price} = \underbrace{w_1}_{\text{parameter}} \times \underbrace{(\text{sqft})}_{\text{feature}} + \underbrace{w_2}_{\text{parameter}} \times \underbrace{(\text{bedrooms})}_{\text{feature}} + \underbrace{w_3}_{\text{parameter}} \times \underbrace{(\text{location score})}_{\text{feature}} + \underbrace{b}_{\text{parameter}}$$

	Parameters	Hyperparameters
What they are	w_1, w_2, w_3, b (the weights and bias)	Learning rate, number of training epochs, regularization strength (λ)
Who sets them	The algorithm , via gradient descent	You, the AI/ML Engineer, before training
When determined	During training, updated every iteration	Before training even begins
Example values	$w_1 = 150$ (₹150 per sqft), $b = 200000$ (base price)	Learning rate = 0.01, Epochs = 100

Hyperparameter Tuning Real-time Examples

- We don't manually decide that "each square foot adds ₹150 to the price" — the model discovers that by looking at thousands of houses.
- But we DO manually decide "let's train for 100 rounds using a learning rate of 0.01" before you even show the model any data.

BEFORE TRAINING (you choose) (Hyperparameters)

Learning rate = 0.01

Epochs = 100

Regularization λ = 0.1

DURING TRAINING (model learns) (Parameters)

—▶ $w1 = 150$ (sqft weight)

—▶ $w2 = 45000$ (bedroom weight)

—▶ $w3 = 8000$ (location weight)

$b = 200000$ (base price)

Hyperparameter Tuning Real-time Examples

HYPERPARAMETERS — you write these before training

```
model = NeuralNetwork(  
    hidden_layers=3,           # hyperparameter  
    neurons_per_layer=64,      # hyperparameter  
    dropout_rate=0.3,          # hyperparameter  
    learning_rate=0.01,        # hyperparameter  
    batch_size=32              # hyperparameter  
)
```

```
model.fit(X_train, y_train, epochs=50)  # epochs = hyperparameter too
```

PARAMETERS — the model produces these; you never write them yourself

```
print(model.weights)  # e.g., [0.42, -1.15, 0.88, ...] ← learned automatically  
print(model.bias)     # e.g., 0.037 ← learned automatically
```

Hyperparameter Tuning Real-time Examples

- **Features** are **what** the model *reads*;
- **Parameters** are **what** the model *learns*;
- **Hyperparameters** are **what** *you decide before* the *model ever sees any data*.

Example 2: Disease Prediction Dataset (e.g., Diabetes/Cancer Classification) Using a neural network to predict disease presence from features like glucose level, blood pressure, BMI, and age:

Parameters (learned automatically)	Hyperparameters (set manually)
Weight connecting "glucose level" neuron to hidden layer	Number of hidden layers (e.g., 3)
Weight connecting "BMI" neuron to hidden layer	Number of neurons per layer (e.g., 64)
Bias term of each neuron	Dropout rate (e.g., 0.3) to prevent overfitting
The final decision boundary weights in the output layer	Batch size (e.g., 32)
—	Choice of activation function (ReLU, sigmoid)
—	Number of training epochs

Neural network hyperparameters

Learning rate

- Learning rate **sets** the **speed** at which a **model** **adjusts** its **parameters** in each **iteration**. These **adjustments** are **known** as **steps**.
- A **high learning rate** means that a model will **adjust more quickly**, but at the **risk** of **unstable performance** and **data drift**.
- Meanwhile, a **low learning rate** is **more time-consuming** and **requires more data**.

Learning rate decay

- Learning rate decay **sets** the **rate** at **which** the **learning rate of a network drops over time**, allowing the **model** to **learn more quickly**.
- An algorithm's **training progression** from its **initial activation** to **ideal performance** is known as **convergence**.

Neural network hyperparameters

- **LEARNING RATE** = How big of a step to take on EACH update (a single number)
- **LEARNING RATE DECAY** = The STRATEGY for shrinking that step size OVER TIME
- Learning rate is a *value*.
- Learning rate decay is a policy/rule that changes that value as training progresses.
- One is a number;
- the other is a function that produces different numbers at different points in training.

Neural network hyperparameters

Batch size

- Batch size sets the **number of samples** the **model** will **compute** before **updating** its **parameters**.
- It has a **significant effect** on **both compute efficiency** and **accuracy** of the **training process**.
- On its own, a **higher batch size weakens overall performance**, but **adjusting the learning rate** along with **batch size** can **mitigate this loss**.

Number of hidden layers

- The **number** of **hidden layers** in a **neural** network **determines** its **depth**, which affects its **complexity** and **learning ability**.
- **Fewer layers** make for a **simpler** and **faster model**, but **more layers** lead to **better classification** of **input data**.
- **Identifying** the **optimal hyperparameter value** here from all the **possible combinations** is all about a **tradeoff** between **speed** with **accuracy**.

Number of nodes or neurons per layer

- The **number** of nodes or **neurons per layer** sets the **width** of the **model**.
- The **more nodes** or **neurons** per **layer**, the greater the breadth of the **model** and the **better** able it is to **depict complex relationships** between data points.

Momentum

- Momentum is the **degree** to which **models update parameters** in the **same direction** as **previous iterations**, rather than **reversing course**.
- Most data scientists **begin with** a **lower hyperparameter** value for **momentum** and then **tweak upwards** as **needed** to **keep the model** on **course** as it **takes in training data**.

Number of nodes or neurons per layer

Epochs

- Epochs is a **hyperparameter** that sets the **number of times** that a **model** is **exposed** to its **entire training dataset** during the **training process**.
- **Greater exposure** can **lead** to **improved performance** **but runs** the **risk** of **overfitting**.

Activation function

- Activation function **introduces nonlinearity** into a **model**, allowing it to **handle more complex datasets**.
- **Nonlinear models** can **generalize** and **adapt** to a **greater variety** of **data**.

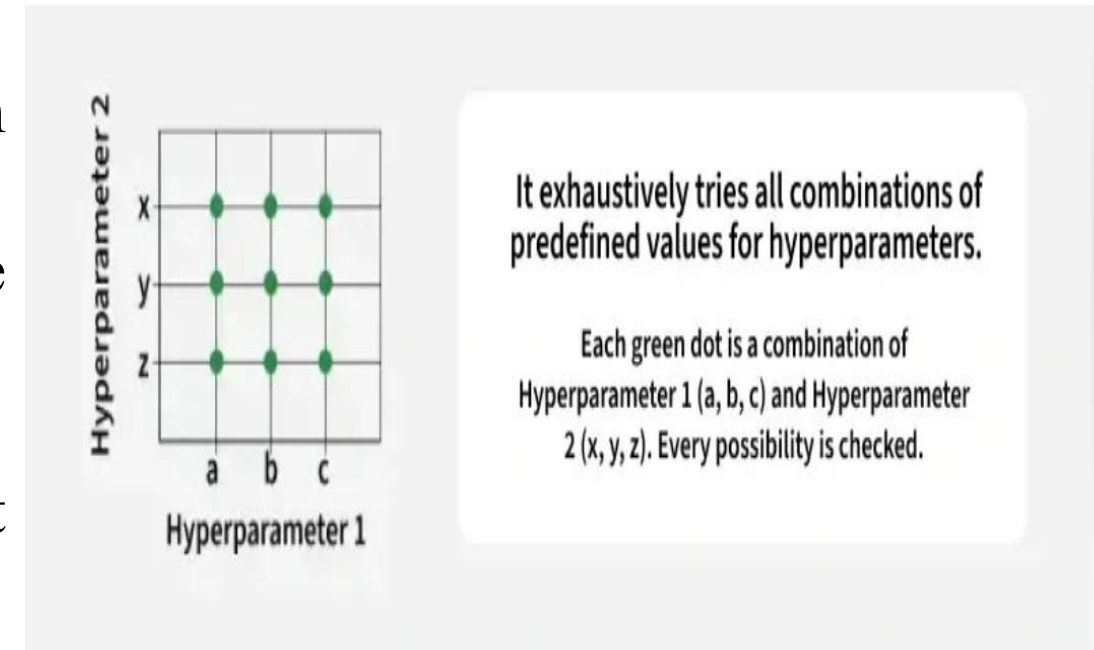
Techniques for Hyperparameter Tuning

- Models can have many hyperparameters, and finding the best combination of parameters can be treated as a search problem.
- The strategies for Hyperparameter tuning are:



Grid Search CV

- It is a **brute-force technique** for **hyperparameter tuning**.
- It **trains the model using all possible combinations of specified hyperparameter values** to **find the best-performing setup**.
- It is **slow** and **uses a lot of computer power** which **makes it hard to use** with **big datasets** or **many settings**.
- It works using below steps:
 - Create a grid of potential values for each hyperparameter.
 - Train the model for every combination in the grid.
 - Evaluate each model using cross-validation.
 - Select the combination that gives the highest score.



Grid Search CV

- For **example**, if we want to **tune two hyperparameters C** and **Alpha** for a **Logistic Regression Classifier** model with the **following sets of values**:

C = [0.1, 0.2, 0.3, 0.4, 0.5]

Alpha = [0.01, 0.1, 0.5, 1.0]

- The grid search technique will construct multiple versions of the model with all possible combinations of C and Alpha, resulting in a total of $5 * 4 = 20$ different models.
- The best-performing combination is then chosen.

0.5	0.701	0.703	0.697	0.696
0.4	0.699	0.702	0.698	0.702
0.3	0.721	0.726	0.713	0.703
0.2	0.706	0.705	0.704	0.701
0.1	0.698	0.692	0.688	0.675
	0.1	0.2	0.3	0.4

Alpha

Example: Tuning Logistic Regression with GridSearchCV

- The following code illustrates how to use GridSearchCV .
- In this below code:
 - We generate **sample data** using **make_classification**.
 - We **define** a range of **C** values using **logarithmic scale**.
 - GridSearchCV tries all combinations from **param_grid** and uses **5-fold cross-validation**.
 - It **returns** the **best hyperparameter (C)** and its **corresponding validation score**

Grid Search CV

```
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import GridSearchCV
import numpy as np
from sklearn.datasets import make_classification
```

```
X, y = make_classification(
    n_samples=1000, n_features=20, n_informative=10, n_classes=2, random_state=42)
```

```
c_space = np.logspace(-5, 8, 15)
```

```
param_grid = {
    'C': c_space,
    'penalty': ['l1', 'l2']
}
```

```
logreg = LogisticRegression(solver='liblinear')
```

```
logreg_cv = GridSearchCV(logreg, param_grid, cv=5)
```

```
logreg_cv.fit(X, y)
```

```
print("Tuned Logistic Regression Parameters: {}".format(logreg_cv.best_params_))
```

```
print("Best score is {}".format(logreg_cv.best_score_))
```

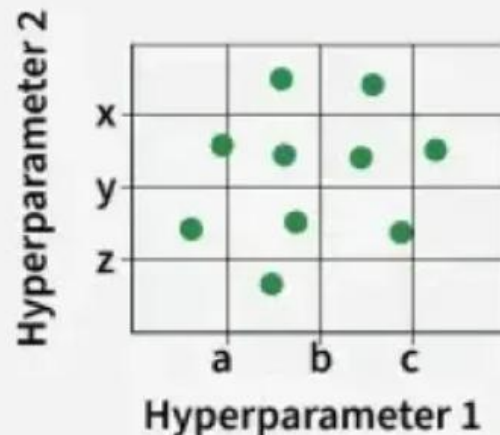
Output:

*Tuned Logistic Regression Parameters: {'C': 0.006105402296585327}
Best score is 0.853*

- This represents the **highest accuracy** achieved by the model using the **hyperparameter combination C = 0.0061**.
- The **best score of 0.853** means the model achieved **85.3% accuracy** on the **validation data** during the **grid search** process.

Randomized Search CV

- As the name suggests it **picks random combinations** of **hyperparameters** from the **given ranges instead** of **checking every single combination** like GridSearchCV.
- In **each iteration** it tries a new random combination of **hyperparameter** values.
- It records the model's performance for **each combination**.
- After several attempts it **selects the best-performing set**.



It randomly picks combinations of hyperparameters instead of trying all.

Green dots are randomly spread in the same parameter space not covering every possible combination.

Example: Tuning Decision Tree with `RandomizedSearchCV`

- The following code illustrates how to use `RandomizedSearchCV`.
- In this example:
- We define a range of values for each **hyperparameter** e.g, `max_depth`, `min_samples_leaf` etc.
- Random combinations are picked and evaluated using **5-fold cross-validation**.
- The **best combination** and **score** are **printed**.


```
import numpy as np
from sklearn.datasets import make_classification

X, y = make_classification(n_samples=1000, n_features=20, n_informative=10, n_classes=2, random_state=42)

from scipy.stats import randint
from sklearn.tree import DecisionTreeClassifier
from sklearn.model_selection import RandomizedSearchCV

param_dist = {
    "max_depth": [3, None],
    "max_features": randint(1, 9),
    "min_samples_leaf": randint(1, 9),
    "criterion": ["gini", "entropy"]
}

tree = DecisionTreeClassifier()
tree_cv = RandomizedSearchCV(tree, param_dist, cv=5)
tree_cv.fit(X, y)

print("Tuned Decision Tree Parameters: {}".format(tree_cv.best_params_))
print("Best score is {}".format(tree_cv.best_score_))
```

Output:

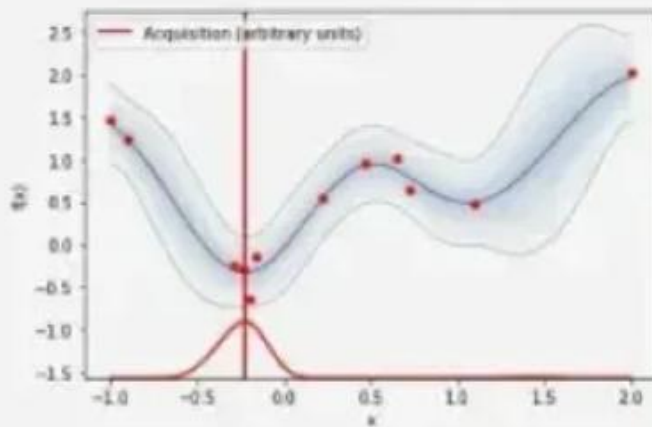
```
Tuned Decision Tree Parameters: {'criterion': 'entropy', 'max_depth': None, 'max_features': 6,
'min_samples_leaf': 6}
Best score is 0.8
```

- A score of **0.842** means the model performed with an accuracy of **84.2%** on the validation set with the mentioned hyperparameters.

Bayesian Optimization

- **Grid Search** and **Random Search** can be **inefficient** because they **blindly try many hyperparameter combinations**, even if some are **clearly not useful**.
- This is a **smarter approach**. It **treats hyperparameter tuning** like a **mathematical optimization problem** and **learns from past results** to **decide what to try next**.
- Build a **probabilistic model** (**surrogate** function) that **predicts performance** based on **hyperparameters**.
- **Update this model after each evaluation**.
- Use the **model** to **choose** the **next best set** to **try**.
- **Repeat until the optimal combination is found**. The **surrogate function** models: $P(\text{score}(y) \mid \text{hyperparameters}(x))$

Bayesian Optimization

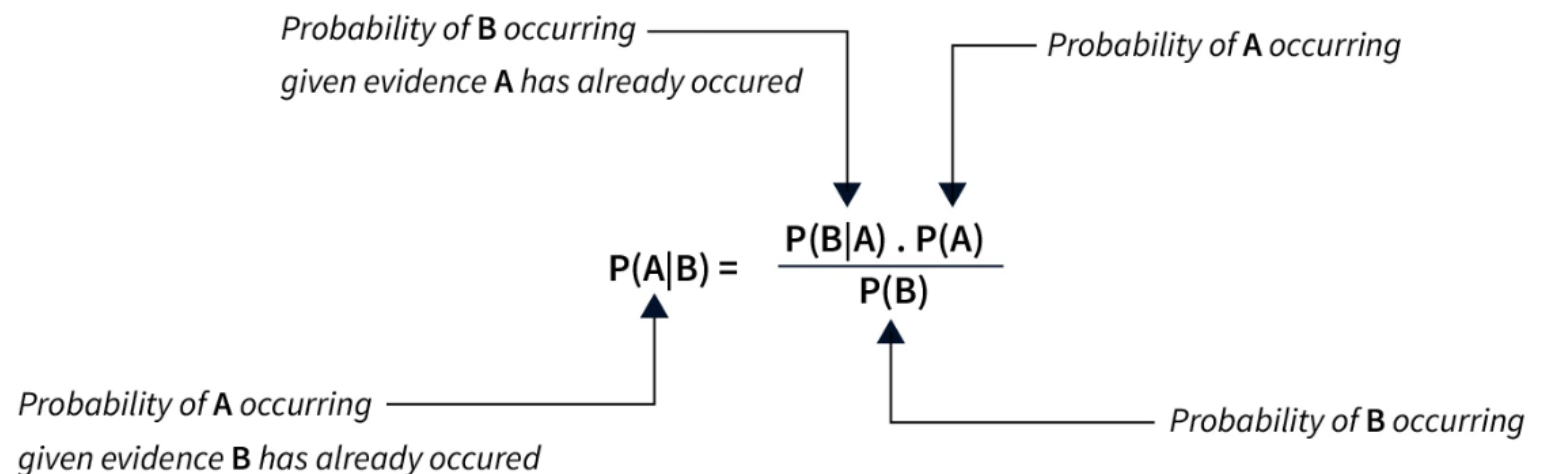


It builds a probabilistic model to predict the performance of hyperparameters and chooses the next best set

The curve shows a prediction model with uncertainty. Red dots are past evaluations. The red vertical line shows the next promising point to try

Bayesian Optimization

- Here the **surrogate function** models the **relationship** between **hyperparameters x** and the **score y** .
- By **updating** this model iteratively with each **new evaluation**, Bayesian optimization makes **more informed decisions**.
- Common **surrogate models** used in **Bayesian optimization** include:
 - Gaussian Processes
 - Random Forest Regression
 - Tree-structured Parzen Estimators (TPE)



The diagram illustrates the relationship between joint, marginal, and conditional probabilities for two events, A and B. It features the equation $P(A|B) = \frac{P(B|A) \cdot P(A)}{P(B)}$ in the center. Four arrows point to the terms in the equation:

- An arrow from the text "Probability of B occurring given evidence A has already occurred" points to $P(B|A)$.
- An arrow from the text "Probability of A occurring" points to $P(A)$.
- An arrow from the text "Probability of A occurring given evidence B has already occurred" points to $P(A|B)$.
- An arrow from the text "Probability of B occurring" points to $P(B)$.

$$P(A|B) = \frac{P(B|A) \cdot P(A)}{P(B)}$$

Advantages of Hyperparameter tuning

- **Improved Model Performance:** Finding the optimal combination of hyperparameters can significantly boost model accuracy and robustness.
- **Reduced Overfitting and Underfitting:** Tuning helps to prevent both overfitting and underfitting resulting in a well-balanced model.
- **Enhanced Model Generalizability:** By selecting hyperparameters that optimize performance on validation data the model is more likely to generalize well to unseen data.

Advantages of Hyperparameter tuning

- **Optimized Resource Utilization:** With **careful tuning resources** such as **computation time** and **memory** can be used **more efficiently** **avoiding unnecessary work**.
- **Improved Model Interpretability:** Properly tuned hyperparameters can make the **model simpler** and **easier** to **interpret**.

Challenges & Solutions in Hyperparameter Tuning

Dealing with High-Dimensional Hyperparameter Spaces:

- The larger the hyperparameter space the more combinations need to be explored. This makes the search process **computationally expensive** and **time-consuming** especially for **complex models** with **many hyperparameters**.

Handling Expensive Function Evaluations:

- **Evaluating a model's performance** can be **computationally expensive**, particularly for models that require a lot of data or iterations.

Challenges & Solutions in Hyperparameter Tuning

Incorporating Domain Knowledge:

- It can help **guide** the **hyperparameter search**, **narrowing down** the **search space** and **making** the **process more efficient**.
- Using **insights** from the **problem context** can **improve** both the **efficiency** and **effectiveness** of **tuning**.

Developing Adaptive Hyperparameter Tuning Methods:

- **Dynamic adjustment** of **hyperparameters** during training such as **learning rate schedules** or **early stopping** can **lead** to **better** model **performance**.

Overfitting and Underfitting

Overfitting and **underfitting** are **two fundamental problems encountered** when training **deep learning models**, both of which **hinder** a **model's ability** to **generalize** to **new, unseen data**.

Underfitting occurs when a **model** is **too simple** to **capture** the **underlying patterns** and **relationships** within the **training data**. It **fails** to **learn effectively** from the **data**.

Characteristics:

- **High error rates** on both the **training data** and **new, unseen data** (validation/test data).
- The model **exhibits high bias**, meaning it **makes strong, incorrect assumptions** about the **data's structure**.

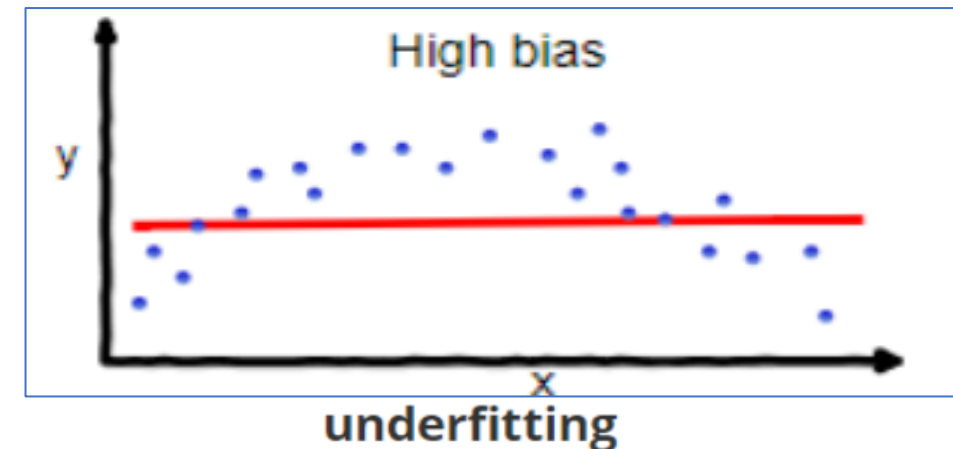
Overfitting and Underfitting

Causes:

- Using a model that is **too simple** for the **complexity** of the **data** (e.g., a linear model on non-linear data).
- **Insufficient training time** or an **inadequate number** of **features**.

Analogy:

- Imagine trying to fit a straight line to data that clearly follows a curved path. The line won't capture the curve's essence.

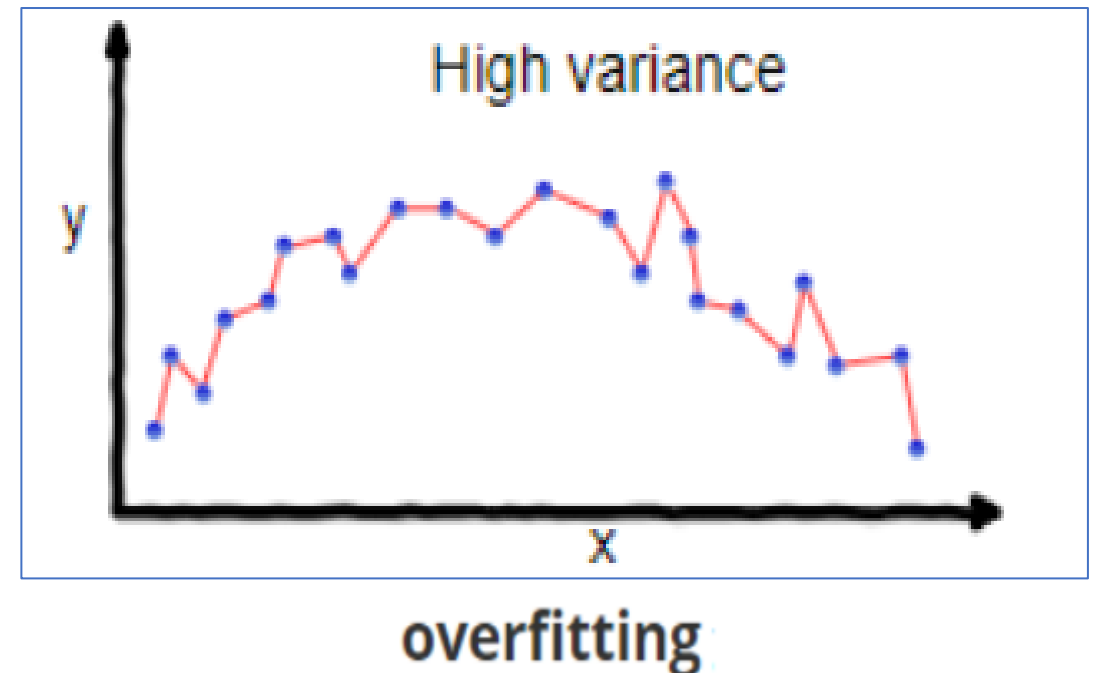


Overfitting and Underfitting

Overfitting occurs when a model learns the training data too well, including its noise and outliers, and consequently fails to generalize to new data. It essentially "memorizes" the training examples rather than learning the underlying concepts.

Characteristics:

- Very low error rates on the training data.
- Significantly higher error rates on new, unseen data (validation/test data).
- The model exhibits high variance, meaning it is overly sensitive to minor fluctuations in the training data.



Overfitting and Underfitting

Causes:

- Using a model that is too complex for the amount or complexity of the data.
- Insufficient training data.
- Lack of regularization techniques to prevent the model from becoming too complex.

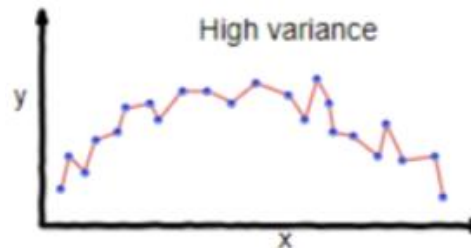
Analogy:

- Imagine a student who memorizes every answer to a practice test but doesn't understand the concepts.
- They might ace the practice test but fail the actual exam if the questions are phrased differently.

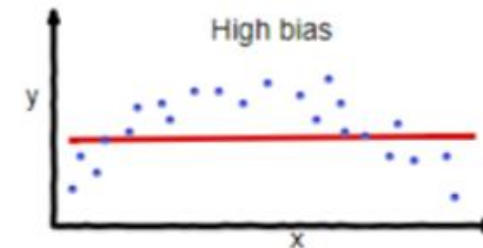
Overfitting and Underfitting

The Goal:

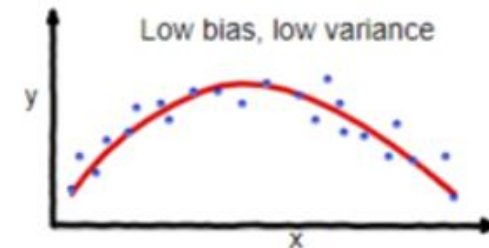
- The aim in deep learning is to **find** a **model** that **achieves** a "**good fit**," **striking** a **balance** between **underfitting** and **overfitting**.
- A good fit means the **model captures** the **essential patterns** in the **data** while **remaining flexible enough** to **generalize well** to **new, previously unseen examples**.
- This typically involves a **low training error** and a **low, comparable validation/test error**.



overfitting



underfitting

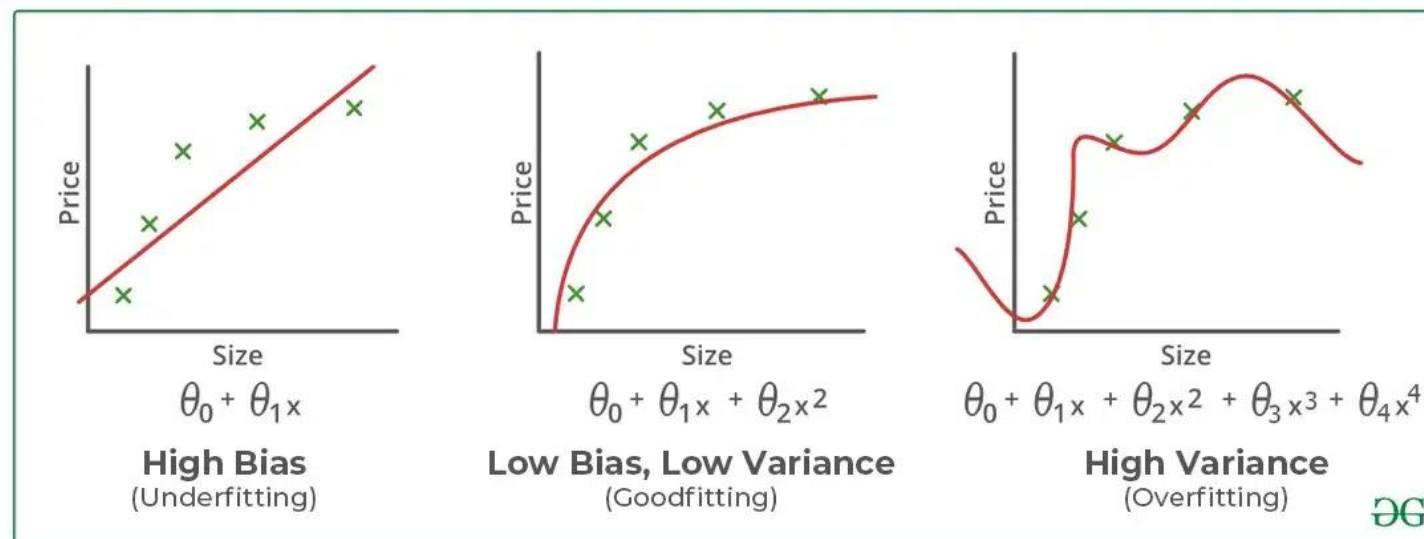


Good balance

Bias and Variance in Deep Learning

- Bias and variance are two key sources of error in deep learning models that directly impact their performance and generalization ability.
- **Bias**: is the error that happens when a deep learning model is too simple and **doesn't learn enough** details **from** the **data**. It's like **assuming all birds** can **only** be **small** and **fly**, so the **model fails** to **recognize big birds** like **ostriches** or **penguins** that **can't fly** and get **biased** with **predictions**.
- These **assumptions** make the **model easier** to **train** but may **prevent** it from **capturing the underlying complexities** of the **data**.
- **High bias** typically **leads** to **underfitting**, where the model performs poorly on both training and testing data because it fails to learn enough from the data.
- **Example**: A **linear regression model** applied to a **dataset** with a **non-linear relationship**.

- **Variance: Error** that **happens** when a **deep learning** model **learns too much** from the data, **including random noise**.
- A high-variance model learns not only the patterns but also the noise in the training data, which leads to **poor generalization** on **unseen data**.
- **High variance** typically leads to **overfitting**, where the model performs **well** on **training** data but **poorly** on **testing data**.



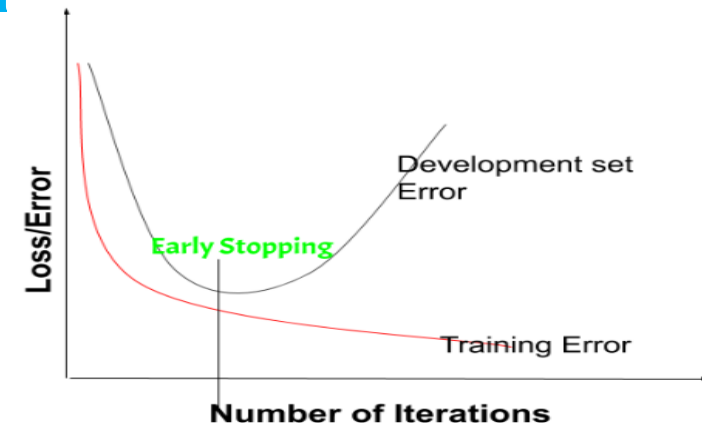
How to Address Overfitting and Underfitting?

Techniques to Reduce **Underfitting**

- **Increase** model **complexity**.
- **Increase** the **number** of **features**, performing feature engineering.
- **Remove noise** from the **data**.
- **Increase** the **number** of epochs **or increase** the **duration** of **training** to get **better results**.

Techniques to Reduce Overfitting

1. Improving the quality of training data reduces overfitting by focusing on meaningful patterns, mitigate the risk of fitting the noise or irrelevant features.
2. Increase the training data can improve the model's ability to generalize to unseen data and reduce the likelihood of overfitting.
3. Reduce model complexity.
4. Early stopping during the training phase (have an eye over the loss over the training period as soon as loss begins to increase stop training)
5. Ridge Regularization and Lasso Regularization.
6. Use dropout for neural networks to tackle overfitting.

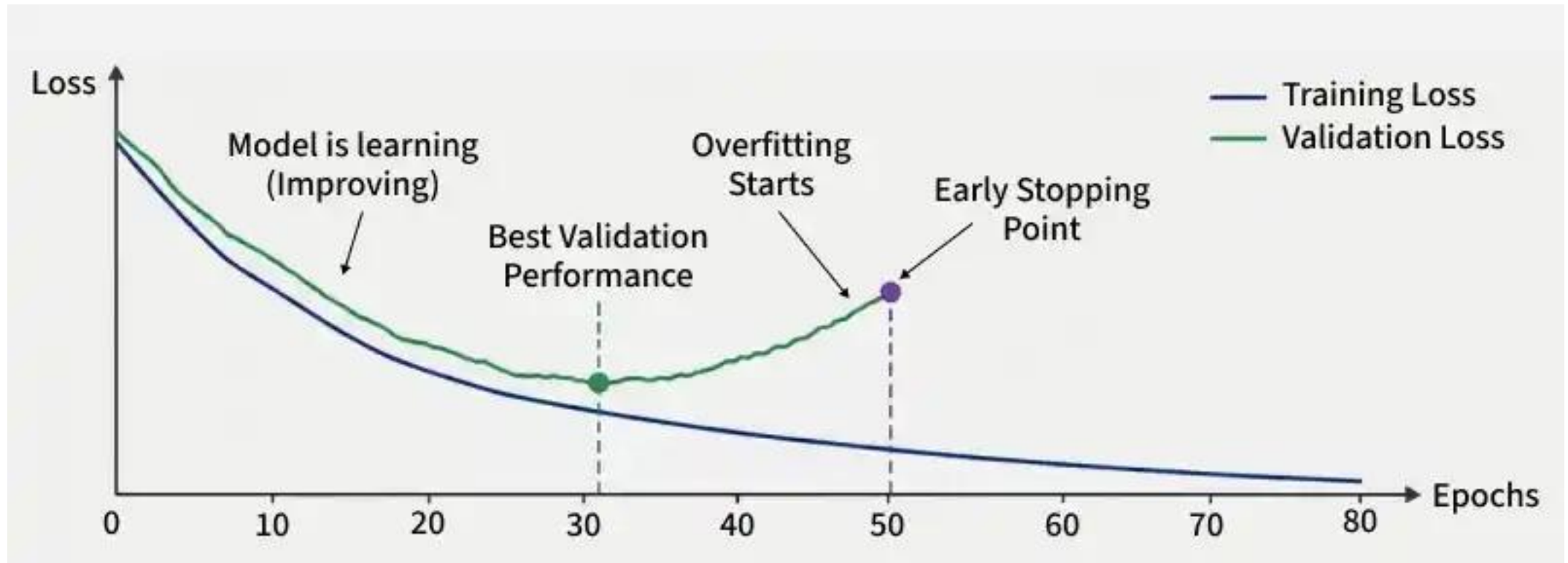


Regularization by Early Stopping

- Regularization techniques are **important** in **deep learning** to **prevent overfitting**, ensuring that **models generalize** well to **unseen data**.
 - One of the most effective and widely-used regularization methods is **early stopping** which aims to **stop training** at the **right time** to **maintain a balance** between **underfitting** and **overfitting**.
- **Early stopping** is a **regularization technique** that **stops** model **training when overfitting signs appear**.
 - It **prevents** the model **from** performing **well** on the **training** set but **underperforming** on **unseen data**, i.e., the **validation set**.
 - **Training stops** when performance **improves** on the **training** set **but degrades** on the **validation** set, **promoting better generalization** while **saving time** and **resources**.

Regularization by Early Stopping

- The technique monitors the model's performance on both the **training** and **validation** sets.
 - If the **validation** performance **worsens**, **training stops** and the **model retains** the **best weights** from the **period** of **optimal validation performance**.



Regularization by Early Stopping

Working

- **Split the Dataset:** Divide the available data into training and validation sets.
 - The training set is used to update the model weights, while the validation set is used only to monitor performance during training.
- **Train the Model:** Train the model for multiple epochs using the training dataset.
- **Evaluate on the Validation Set:** At the end of every epoch, calculate the selected validation metric, such as validation loss or validation accuracy.

Regularization by Early Stopping

Working

- **Compare with the Best Performance:** If the **validation metric improves**, **save** the **current** model **weights** as the **best-performing** model and **reset** the **patience counter**.
- **Check the Patience Value:** If the **validation metric** does not improve for the **specified number of consecutive epochs**, the **patience counter** reaches its **limit** and **training is stopped automatically**.
- **Restore the Best Model:** If **enabled**, the model **restores** the **weights** from the **epoch** that **achieved** the **best validation performance** before **completing training**.

Regularization by Early Stopping

Key Parameters

These **parameters** can be **configured** using the **built-in** Early Stopping **callback available** in deep learning frameworks such as TensorFlow and Keras.

Parameter	Description
monitor	Specifies the metric to track during training , such as val_loss or val_accuracy . Training decisions are based on changes in this metric .
patience	Defines the number of consecutive epochs to wait for an improvement in the monitored metric before stopping training .

Regularization by Early Stopping

Parameter	Description
min_delta	Sets the minimum change in the monitored metric that qualifies as an improvement . Smaller changes than this value are ignored .
mode	Determines whether the monitored metric should be minimized ("min"), maximized ("max") or inferred automatically ("auto").
restore_best_weights	If enabled (True), restores the model weights from the epoch that achieved the best validation performance after training stops .

Regularization by Early Stopping

Implementation using TensorFlow/Keras

In this implementation, we will train a simple neural network on the **MNIST handwritten digit dataset** and apply **Early Stopping** to automatically stop training when the **validation loss stops improving**.

Step 1: Import Required Libraries

First, **import** the required **libraries** for **building** the **neural network**, **loading** the **dataset** and **visualizing** the **training process**.

```
import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Flatten, Dense
from tensorflow.keras.callbacks import EarlyStopping
import matplotlib.pyplot as plt
```

Regularization by Early Stopping

Implementation using TensorFlow/Keras

Step 2: Load and Preprocess the Dataset: Load the MNIST dataset and normalize the pixel values to the range [0, 1] for faster and more stable training.

```
(x_train, y_train), (x_test, y_test) = tf.keras.datasets.mnist.load_data()
```

```
x_train = x_train / 255.0  
x_test = x_test / 255.0
```

Step 3: Build the Neural Network: Create a simple fully connected neural network for handwritten digit classification.

```
model = Sequential([  
    Flatten(input_shape=(28, 28)),  
    Dense(128, activation="relu"),  
    Dense(10, activation="softmax")  
)  
model.compile(  
    optimizer="adam",  
    loss="sparse_categorical_crossentropy",  
    metrics=["accuracy"]  
)
```


Regularization by Early Stopping

Implementation using TensorFlow/Keras

Step 4: Create the Early Stopping Callback

Configure the Early Stopping callback to monitor the validation loss. Training will stop if the validation loss does not improve for 3 consecutive epochs and the best model weights will be restored.

- `monitor="val_loss"` monitors the validation loss after each epoch.
- `patience=3` allows training to continue for three additional epochs without improvement.
- `restore_best_weights=True` restores the model weights from the epoch with the lowest validation loss.

```
early_stopping = EarlyStopping(  
    monitor="val_loss",  
    patience=3,  
    restore_best_weights=True)
```

Regularization by Early Stopping

Implementation using TensorFlow/Keras

Step 5: Train the Model with Early Stopping

Train the model while reserving 20% of the training data for validation.

```
history = model.fit(  
    x_train,  
    y_train,  
    epochs=20,  
    batch_size=32,  
    validation_split=0.2,  
    callbacks=[early_stopping],  
    verbose=1  
)
```

Regularization by Early Stopping

Implementation using TensorFlow/Keras

Step 6: Evaluate the Model: Evaluate the trained model on the test dataset.

```
test_loss, test_accuracy = model.evaluate(x_test, y_test)

print(f"Test Loss: {test_loss:.4f}")
print(f"Test Accuracy: {test_accuracy:.4f}")
```

Output:

313/313 —————1s 2ms/step - accuracy: 0.9766 - loss: 0.0775

Test Loss: 0.0775

Test Accuracy: 0.9766

Regularization by Early Stopping

Implementation using TensorFlow/Keras

Step 7: Visualize the Training Process: Plot the training loss and validation loss to observe where Early Stopping terminated the training.

```
plt.figure(figsize=(8, 5))

plt.plot(history.history["loss"], label="Training Loss")
plt.plot(history.history["val_loss"], label="Validation Loss")

plt.xlabel("Epoch")
plt.ylabel("Loss")
plt.title("Training and Validation Loss")
plt.legend()

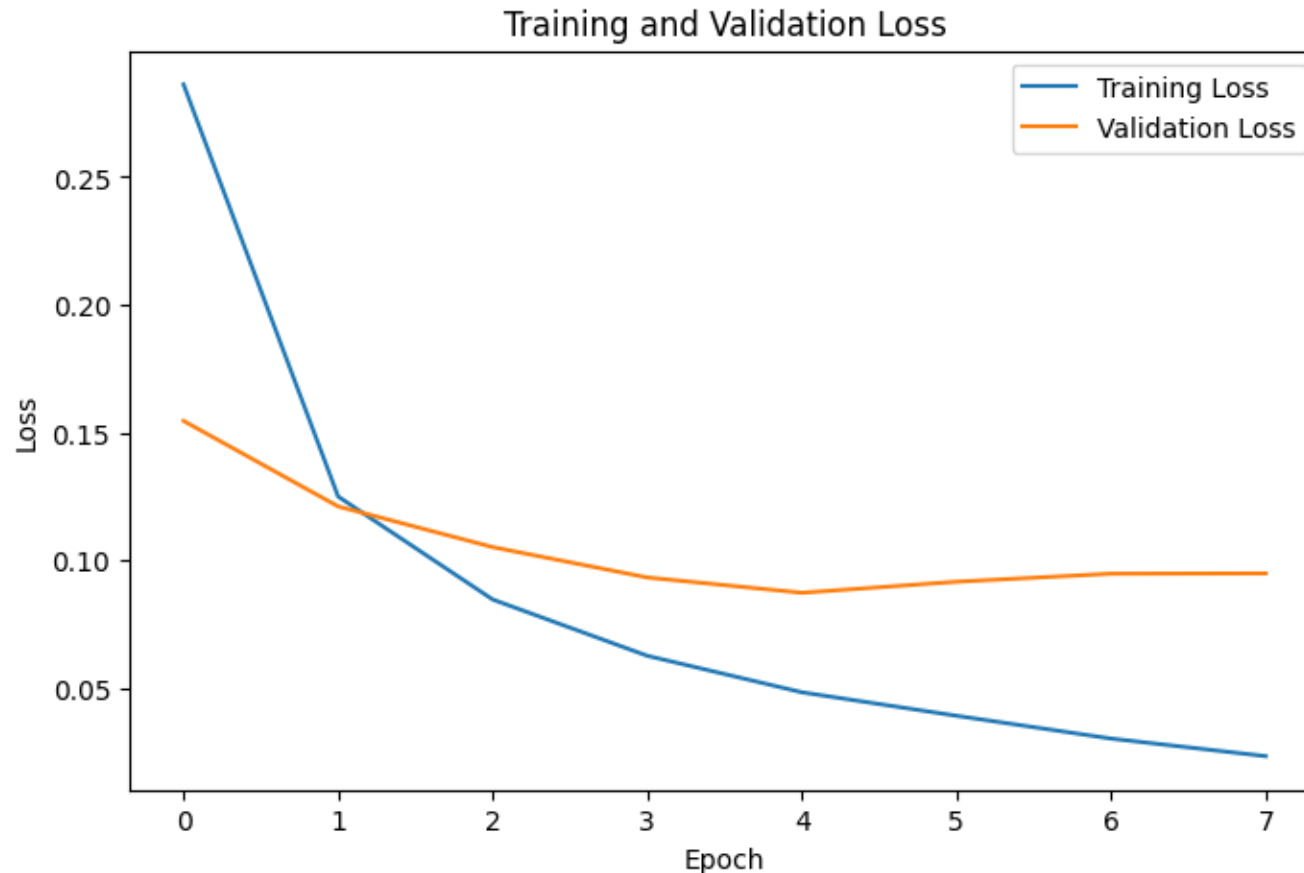
plt.show()
```

Regularization by Early Stopping

Implementation using TensorFlow/Keras

Step 7: Visualize the Training Process: Plot the training loss and validation loss to observe where Early Stopping terminated the training.

Output:

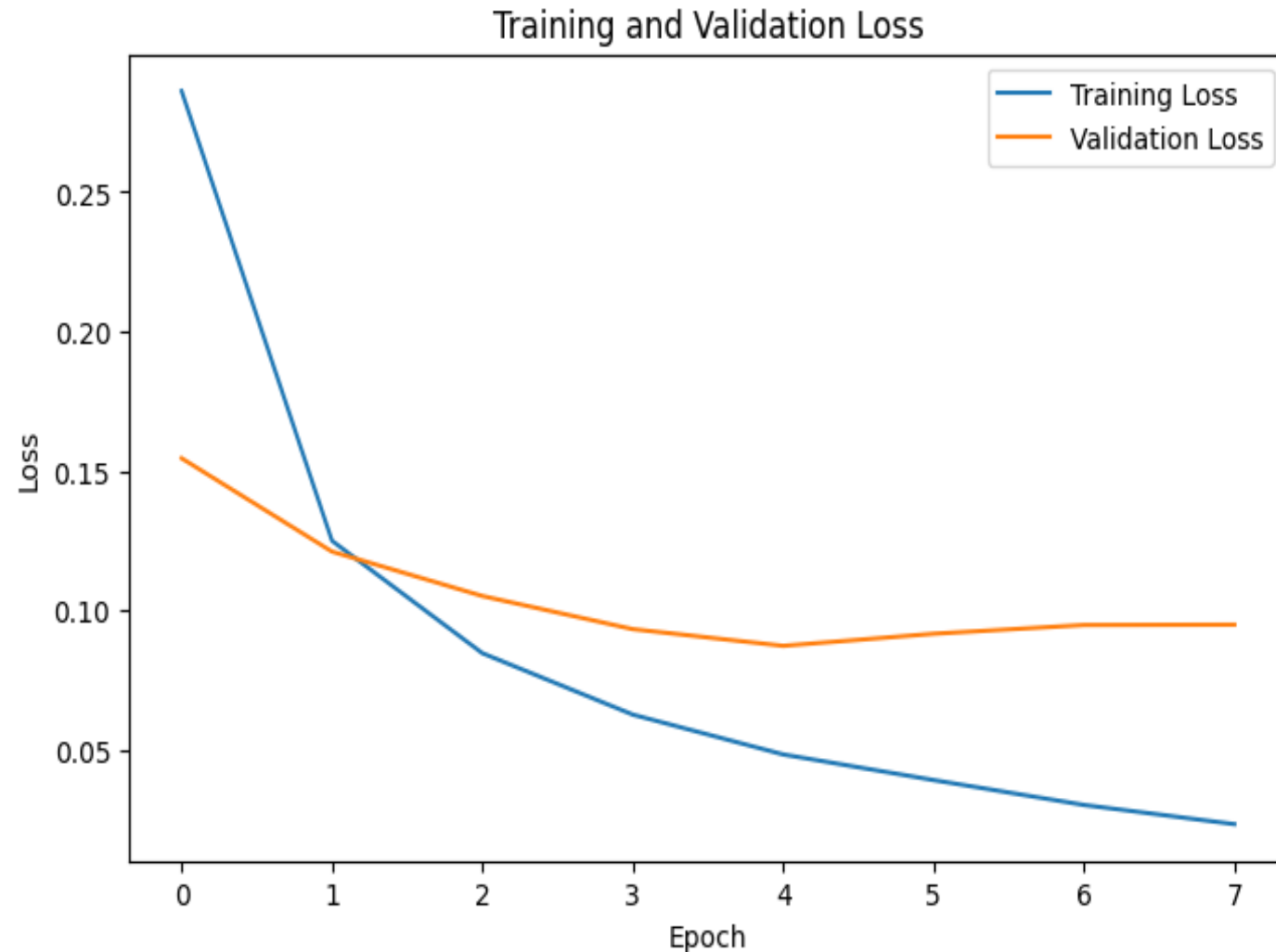


Regularization by Early Stopping

Implementation using TensorFlow/Keras

The graph shows that:

- The **training loss** continuously decreases with each epoch.
- The **validation loss decreases initially but eventually stops improving.**
- After the validation loss fails to improve for three consecutive epochs, **Early Stopping** terminates the training automatically.



Regularization by Early Stopping

Applications

- **Medical Image Diagnosis:** Stops training before medical imaging models memorize limited patient scans, improving diagnosis on unseen cases.
- **Image Classification:** Prevents CNNs from overfitting large image datasets while maintaining high recognition accuracy.
- **Natural Language Processing:** Helps language models generalize better when trained on limited text corpora.
- **Fraud Detection:** Improves detection of unseen fraudulent transactions by preventing memorization of historical patterns.
- **Time Series Forecasting:** Produces forecasting models that generalize better to future observations instead of historical noise.

Regularization by Early Stopping

Advantages

- Prevents overfitting by selecting the model before validation performance degrades.
- Reduces training time by avoiding unnecessary epochs.
- Saves computational resources during deep learning training.
- Works with almost any neural network architecture.
- Easy to implement using built-in callbacks in TensorFlow and Keras.

Limitations

- Choosing an inappropriate patience value can lead to underfitting or overfitting.
- Requires a separate validation dataset, reducing available training data.
- Validation metrics may fluctuate, causing inconsistent stopping decisions.

Regularization by Lasso, Ridge and Elastic Net

- These **regularization techniques** are **universal mathematical concepts**, not techniques locked to one field. Usually called by their **generic mathematical names** ("L1 regularization," "L2 regularization," or "weight decay") rather than "Lasso"/"Ridge."
- They **apply anywhere** you have **learnable weights/parameters** — which **includes classical ML and deep learning**.

Lasso (L1), Ridge (L2), Elastic Net = Ways of PENALIZING large weights in the LOSS FUNCTION

This **applies to ANY model** with **learnable weights**:

- **Linear Regression (ML)**
- **Logistic Regression (ML)**
- **Neural Networks (Deep Learning)**
- **CNNs, RNNs, Transformers (Deep Learning)**

Regularization by Lasso, Ridge and Elastic Net

Term	Original Context	Modern Reality
"Ridge Regression"	A linear regression model with L2 penalty (Hoerl & Kennard, 1970)	The underlying L2 penalty is now used everywhere, including deep learning (often just called "weight decay")
"Lasso Regression"	A linear regression model with L1 penalty (Tibshirani, 1996)	The underlying L1 penalty is used in neural networks too, though less commonly than L2
"Elastic Net"	Combined L1+L2 penalty, originally for linear regression (Zou & Hastie, 2005)	The combined penalty concept applies to neural network weights as well

L1 and L2 Regularization Techniques in Linear Regression (and other ML Models)

- **L1 (LASSO-Least Absolute Shrinkage and Selection Operator) and L2 (Ridge) Regularization** are fundamental techniques used in Machine Learning to prevent overfitting by adding a penalty term to the model's loss function, forcing the model to learn smaller, more generalized weights.
- **LASSO Regression (L1 Regularization)** technique, known as LASSO was introduced much later by **Robert Tibshirani** in **1996**. This technique was specifically developed to **produce sparse models** by driving **some coefficients exactly to zero**, thereby **performing automatic feature selection**. LASSO became a powerful tool, **particularly in high-dimensional datasets** where **feature selection is critical**.

Ridge Regression (L2 Regularization):

- The idea of adding a **squared penalty** to the **least squares objective function** was formally introduced by **Arthur E. Hoerl** and **Robert W. Kennard** in **1970** in their **paper** "**Ridge Regression: *Biased Estimation for Nonorthogonal Problems.***"
- Their primary **motivation** was to **handle multicollinearity (highly correlated predictors)** in **linear regression**, which leads to unstable and high-variance estimates of the coefficients.
- **L2 regularization** is also a form of **Tikhonov regularization**, which was **developed independently for solving ill-posed problems.**

L1 and L2 Regularization Techniques in Linear Regression (and other ML & DL Models)

- They help **improve** generalisation and **model stability**.
- The **strength** of this **penalty** is **controlled** by the **hyperparameter** “**lambda**”.
- Regularisation (L1 / L2):
 - **L1** (Lasso) → **Adds $|w|$ penalty**
 - **L2** (Ridge) → **Adds w^2 penalty**

L1 and L2 Regularization Techniques in Linear Regression (and other ML & DL Models)

- **L1 Regularization:**

- **(Lasso)** Adds the **absolute** value of **weights** as a **penalty**
- **Encourages sparsity** by forcing some **weights** to **become exactly zero**
- **Performs implicit feature selection** (*automatic feature pruning by the model that is, we do not explicitly choose features like using correlation or PCA, the algorithm learns to ignore them during optimisation*)

Loss Function:

$$\text{Loss} = \text{Error} + \lambda \sum |w|$$

L1 and L2 Regularization Techniques in Linear Regression (and other ML & DL Models)

- **Key Characteristics:**
 - Produces **simpler, interpretable** models
 - **Useful** when **many features** are **irrelevant**
 - **Sensitive to correlated features**

L1 and L2 Regularization Techniques in Linear Regression (and other ML & DL Models)

- **L2 Regularization (Ridge)**
 - Adds the **square of weights** as a **penalty**
 - **Shrinks weights** toward **zero** but **does** not **eliminate** them
 - **Reduces model variance**

Loss Function:

$$\text{Loss} = \text{Error} + \lambda \sum w^2$$

L1 and L2 Regularization Techniques in Linear Regression (and other ML & DL Models)

- **L2 Regularization (Ridge)-Key Characteristics:**
 - Handles **multicollinearity** well
 - Produces **more stable models**
 - **Keeps all features** with **small weights**
- **When to Use What**
 - Use **L1** when **feature selection** is required
 - Use **L2** when **features are correlated**, and **stability** is **important**
 - Use Elastic Net to combine both benefits

L1 and L2 Regularization Techniques in Linear Regression (and other ML & DL Models)

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3 import pandas as pd # Added for a cleaner 'head' display
4 from sklearn.linear_model import Lasso, Ridge
5 from sklearn.datasets import fetch_california_housing
6 from sklearn.preprocessing import StandardScaler
7
8 # 1. Load Data
9 data = fetch_california_housing()
10 X, y = data.data, data.target
11 feature_names = data.feature_names
12
```

L1 and L2 Regularization Techniques in Linear Regression (and other ML & DL Models)

```
13 # --- DATA INSPECTION ---
14 print("--- Data Inspection ---")
15 # Displaying the number of rows and columns
16 print(f"Dataset Shape: {X.shape[0]} rows and {X.shape[1]} columns")
17
18 # Display the first five rows (using a DataFrame for pretty printing)
19 df_preview = pd.DataFrame(X, columns=feature_names)
20 print("\nFirst 5 rows of features:")
21 print(df_preview.head())
22 print("-" * 25)
23
```

L1 and L2 Regularization Techniques in Linear Regression (and other ML & DL Models)

```
24 # 2. Scale Features (Essential for fair comparison)
25 scaler = StandardScaler()
26 X_scaled = scaler.fit_transform(X)
27
28 # 3. Define a range of Alphas (Lambda)
29 alphas = np.logspace(-3, 2, 100)
30
31 lasso_coefs = []
32 ridge_coefs = []
33
```

L1 and L2 Regularization Techniques in Linear Regression (and other ML & DL Models)

```
34 for a in alphas:
35     lasso = Lasso(alpha=a, max_iter=10000)
36     lasso.fit(X_scaled, y)
37     lasso_coefs.append(lasso.coef_)
38
39     ridge = Ridge(alpha=a)
40     ridge.fit(X_scaled, y)
41     ridge_coefs.append(ridge.coef_)
42
43 # 4. Plotting
44 plt.figure(figsize=(16, 7))
45
```

L1 and L2 Regularization Techniques in Linear Regression (and other ML & DL Models)

```
45
46 # --- L1 (Lasso) Plot ---
47 plt.subplot(1, 2, 1)
48 plt.plot(alphas, lasso_coefs)
49 plt.xscale('log')
50 plt.title('L1 (Lasso): Feature Selection\n(Lines hitting 0.0 are excluded)', fontsize=14)
51 plt.xlabel('Regularization Strength (Alpha)', fontsize=12)
52 plt.ylabel('Coefficient Value (Weight)', fontsize=12)
53 plt.legend(feature_names, loc='upper right', fontsize='8')
54 plt.grid(True, alpha=0.3)
55
```

L1 and L2 Regularization Techniques in Linear Regression (and other ML & DL Models)

```
56 # --- L2 (Ridge) Plot ---
57 plt.subplot(1, 2, 2)
58 plt.plot(alphas, ridge_coefs)
59 plt.xscale('log')
60 plt.title('L2 (Ridge): Weight Shrinkage\n(Lines approach 0 but stay active)', fontsize=14)
61 plt.xlabel('Regularization Strength (Alpha)', fontsize=12)
62 plt.ylabel('Coefficient Value (Weight)', fontsize=12)
63 plt.legend(feature_names, loc='upper right', fontsize='8')
64 plt.grid(True, alpha=0.3)
65
66 plt.tight_layout()
67 plt.show()
```

L1 and L2 Regularization Techniques in Linear Regression (and other ML & DL Models)

--- Data Inspection ---

Dataset Shape: 20640 rows and 8 columns

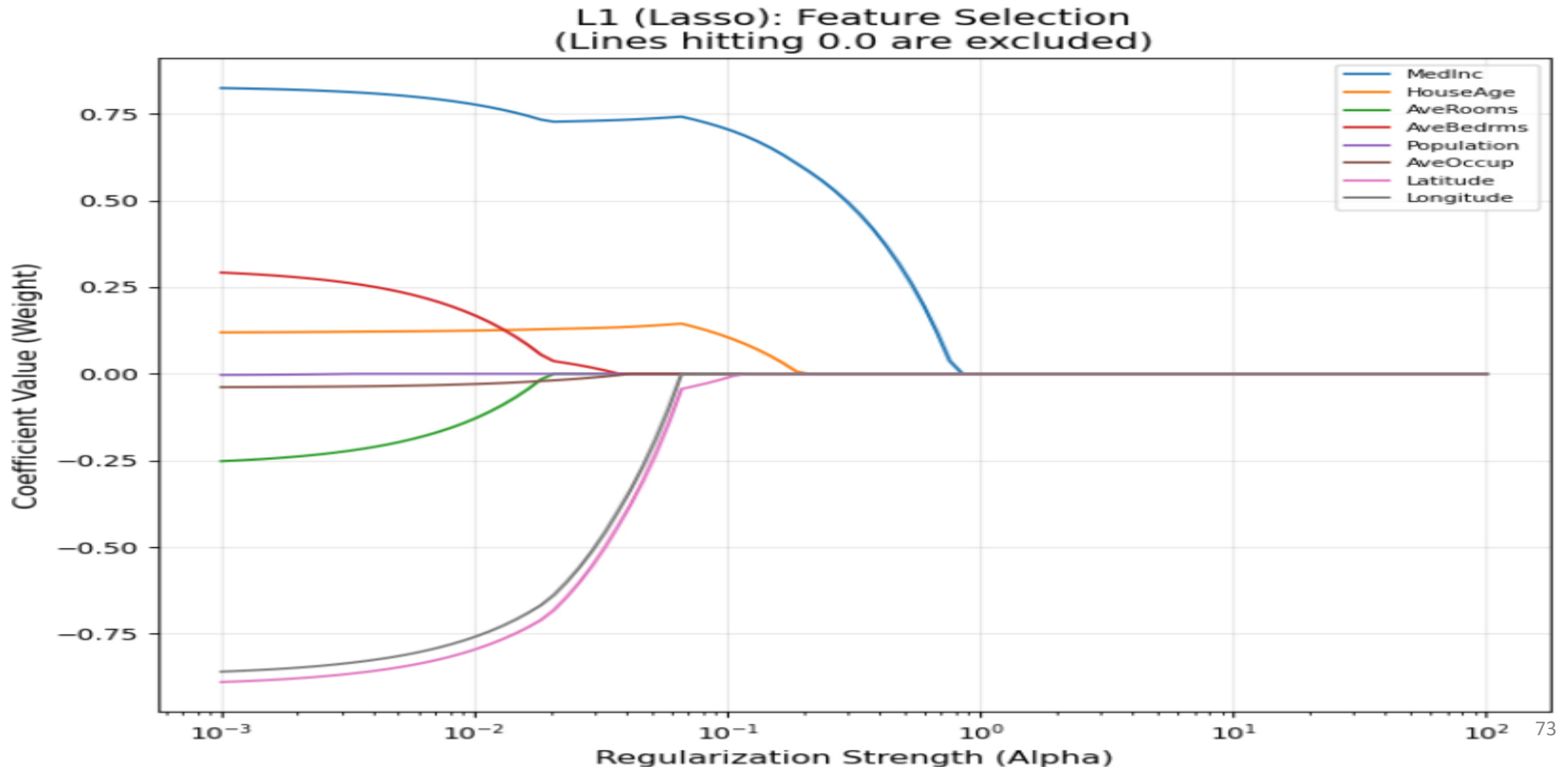
First 5 rows of features:

	MedInc	HouseAge	AveRooms	AveBedrms	Population	AveOccup	Latitude	\
0	8.3252	41.0	6.984127	1.023810	322.0	2.555556	37.88	
1	8.3014	21.0	6.238137	0.971880	2401.0	2.109842	37.86	
2	7.2574	52.0	8.288136	1.073446	496.0	2.802260	37.85	
3	5.6431	52.0	5.817352	1.073059	558.0	2.547945	37.85	
4	3.8462	52.0	6.281853	1.081081	565.0	2.181467	37.85	

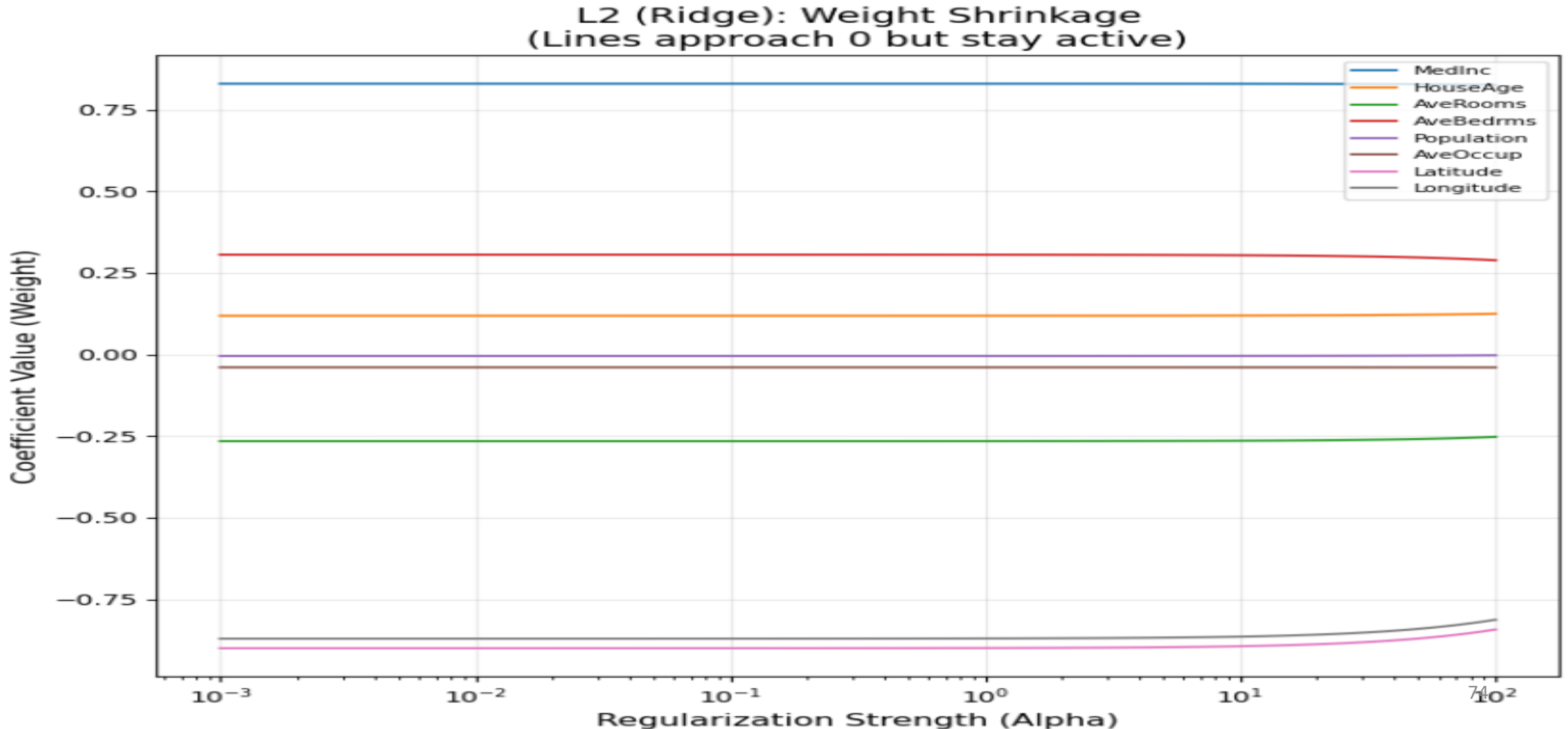
	Longitude
0	-122.23
1	-122.22
2	-122.24
3	-122.25
4	-122.25

-----7/1/2025-----

L1 and L2 Regularization Techniques in Linear Regression (and other ML & DL Models)



L1 and L2 Regularization Techniques in Linear Regression (and other ML & DL Models)



Regularization by Lasso, Ridge and Elastic Net

Ridge Regression (L2 Regularization)

Formula :

$$RidgeLoss = \sum_{i=1}^m (y_i - \hat{y}_i)^2 + \lambda \sum_{j=1}^n \beta_j^2$$

Where,

- *The first term calculates the prediction error.*
- *The second term penalizes large coefficients controlled by λ .*

Regularization by Lasso, Ridge and Elastic Net

Example: Let's assume we are predicting house prices with features like size, location and number of rooms. The model might give coefficients like:

- $\beta_1 = 5$ (Size coefficient)
- $\beta_2 = 3$ (Number of rooms coefficient)
- $\lambda = 0.1$ (regularization strength).

The penalty term for Ridge would be calculated as:

$$\lambda (\beta_1^2 + \beta_2^2) = 0.1 \cdot (5^2 + 3^2) = 0.1 \cdot (25 + 9) = 0.1 \cdot 34 = 3.4$$

This penalty shrinks the coefficients to reduce overfitting but does not remove any features.

Effect:

- Shrinks coefficients towards zero but **never exactly zero**.
- Reduces variance, making the model more stable in the presence of multicollinearity.
- Best when **all features are useful** but we want to control their influence.

➤ implementing using python:

- **ridge = Ridge(alpha=1.0):** Creates a Ridge regression model with regularization strength alpha set to 1.0.

```
from sklearn.linear_model import Ridge
from sklearn.datasets import make_regression
from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_squared_error
```

```
X, y = make_regression(n_samples=100, n_features=5, noise=0.1, random_state=42)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
```

```
ridge = Ridge(alpha=1.0)
ridge.fit(X_train, y_train)
y_pred = ridge.predict(X_test)

mse = mean_squared_error(y_test, y_pred)
print("Mean Squared Error:", mse)
print("Coefficients:", ridge.coef_)
```

Output:

Mean Squared Error: 4.114050771972589

Coefficients: [59.87954432 97.15091098 63.24364738 56.31999433 35.34591136]

- The output shows the **MSE** showing **model performance**. **Lower MSE means better accuracy.**
- The **coefficients** reflect the **regularized feature weights**.

Regularization by Lasso, Ridge and Elastic Net

Lasso Regression (L1 Regularization)

$$LassoLoss = \sum_{i=1}^m (y_i - \hat{y}_i)^2 + \lambda \sum_{j=1}^n |\beta_j|$$

where:

- *The first term calculates the prediction error.*
- *The second term encourages sparsity by shrinking some coefficients to zero.*

Regularization by Lasso, Ridge and Elastic Net

Example: Let's assume the same house price prediction example but now using Lasso. Assume:

- $\beta_1 = 5$ (Size coefficient)
- $\beta_2 = 0$ (Number of rooms coefficient is irrelevant and should be removed)
- $\lambda = 0.1$ (regularization strength).

The penalty term for Lasso would be:

$$\lambda \cdot |\beta_1| = 0.1 \cdot |5| = 0.1 \cdot 5 = 0.5$$

Here Lasso forces $\beta_2 = 0$ removing the Number of Rooms feature entirely from the model.

Effect:

- Can shrink some coefficients **exactly to zero**, effectively performing **feature selection**.
- Helps simplify the model by removing irrelevant or redundant predictors.
- Useful when we believe only a subset of features are truly important.

Example: implementing using python:

- `X, y = make_regression(n_samples=100, n_features=5, noise=0.1, random_state=42)`: **Generates a regression dataset** with **100 samples, 5 features and some noise**.
- `X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)`: **Splits the data into 80% training and 20% testing sets**.
- `lasso = Lasso(alpha=0.1)`: **Creates a Lasso regression model** with **regularization strength alpha set to 0.1**.


```
from sklearn.linear_model import Lasso
from sklearn.model_selection import train_test_split
from sklearn.datasets import make_regression
from sklearn.metrics import mean_squared_error
```

```
X, y = make_regression(n_samples=100, n_features=5, noise=0.1, random_state=42)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
```

```
lasso = Lasso(alpha=0.1)
lasso.fit(X_train, y_train)
```

```
y_pred = lasso.predict(X_test)
```

```
mse = mean_squared_error(y_test, y_pred)
print(f"Mean Squared Error: {mse}")
```

```
print("Coefficients:", lasso.coef_)
```

Output:

Mean Squared Error: 0.06362439921332456

Coefficients: [60.50305581 98.52475354 64.3929265 56.96061238 35.52928502]

The **output** shows the **model's prediction error** and the **importance of features** with some **coefficients reduced to zero** due to **L1 regularization**.

Regularization by Lasso, Ridge and Elastic Net

Elastic Net Regression (L1 + L2 Regularization)

$$ElasticNetLoss = \sum_{i=1}^m (y_i - \hat{y}_i)^2 + \lambda_1 \sum_{j=1}^n |\beta_j| + \lambda_2 \sum_{j=1}^n \beta_j^2$$

where:

- *The first term calculates the prediction error.*
- *The second term applies the L1 penalty for feature selection.*
- *The third term applies the L2 penalty to handle multicollinearity.*
- *It provides a more stable and generalizable model compared to using Lasso or Ridge alone.*

Regularization by Lasso, Ridge and Elastic Net

Example: Let's assume we are predicting house prices using Size and Number of Rooms. Assume:

- $\beta_1 = 5$ (Size coefficient)
- $\beta_2 = 3$ (Number of rooms coefficient)
- $\lambda_1 = 0.1$ (L1 regularization).
- $\lambda_2 = 0.1$ (L2 regularization).

The penalty term for Elastic Net would be:

$$\lambda_1 \cdot (|\beta_1| + |\beta_2|) + \lambda_2 \cdot (\beta_1^2 + \beta_2^2) = 0.1 \cdot (|5| + |3|) + 0.1 \cdot (5^2 + 3^2) = 0.1 \cdot (5 + 3) + 0.1 \cdot (25 + 9) = 0.1 \cdot 8 + 0.1 \cdot 34 = 0.8 + 3.4 = 4.2$$

This penalty shrinks both coefficients but because of the mixture of L1 and L2 it does not force any feature to zero unless absolutely necessary.

Effect:

- Balances **feature selection** (from Lasso) and **coefficient stability** (from Ridge).
- Particularly useful when there are **many correlated predictors**.

Implementation using Python:

- `model = ElasticNet(alpha=1.0, l1_ratio=0.5)` : Creates an Elastic Net model with regularization strength `alpha=1.0` and L1/L2 mixing ratio `0.5`.

```
from sklearn.linear_model import ElasticNet
from sklearn.datasets import make_regression
from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_squared_error
```

```
X, y = make_regression(n_samples=100, n_features=10, noise=0.1, random_state=42)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
```

```
model = ElasticNet(alpha=1.0, l1_ratio=0.5)
model.fit(X_train, y_train)
```

```
y_pred = model.predict(X_test)
mse = mean_squared_error(y_test, y_pred)
```

```
print("Mean Squared Error:", mse)
print("Coefficients:", model.coef_)
```

Output:

Mean Squared Error: 7785.886176938014

Coefficients: [16.84528938 31.77080959 4.05901996 40.18486737 57.25856154 45.81463318
58.97979422 -0. 3.82816854 41.1096051]

- The **output** shows **MSE** which measures how far off predictions are from **actual values** (lower is better) and **coefficients** show feature importance.

Regularization by Lasso, Ridge and Elastic Net

- There's no single “closed-form” formula for optimal λ . Instead, we usually pick it by cross-validation:
- Choose a range of λ values (e.g., from 10^{-4} to 10^3 , spaced logarithmically).
- Split your dataset into k folds (e.g., 5-fold CV).
- For each λ candidate:
 - Train the model on $k-1$ folds.
 - Evaluate Mean Squared Error (or another metric) on the remaining fold.
 - Repeat for all folds and average the error.
- Pick the λ with the **lowest average validation error**.
- Retrain the model on the **entire training set** using this λ .

Regularization by Lasso, Ridge and Elastic Net

Features	Lasso Regression	Ridge Regression	Elastic Net Regression
Penalty Type	L1 Penalty: Lasso uses the absolute values of coefficients.	L2 Penalty: Ridge uses the square of the coefficients.	L1 + L2 Penalty: Elastic Net uses both absolute and square penalties together.
Effect on Coefficients	It completely removes unnecessary features by setting their coefficients to zero.	It makes all coefficients smaller but doesn't set them to zero.	It removes some features and reduces others by balancing both.
Best Use Case	It is best when we want to remove irrelevant features	It is good when all features matter but we want to reduce their impact	It is best for when we features are correlated and feature selection is needed
Hyperparameters involved	Alpha: Controls how much regularization is applied. A higher alpha means more shrinkage.	Alpha: Similar to Lasso which helps in controlling the strength of regularization.	Alpha + L1_ratio: Two parameters. Alpha controls regularization strength and L1_ratio adjusts the balance between Lasso and Ridge.

Regularization by Lasso, Ridge and Elastic Net

Features	Lasso Regression	Ridge Regression	Elastic Net Regression
Bias and Variance	High bias, low variance	Low bias, high variance	Balance of bias and variance
Strengths	It is great for automatically choosing important features.	It works well when features are related but shouldn't be completely removed.	It combines Lasso's feature selection and Ridge's handling of correlations.
Weaknesses	It can sometimes remove useful features if not tuned properly.	It keeps all features which may not help in high-dimensional data with irrelevant features.	It is a bit harder to tune due to having two parameters.
Example	Imagine we have 100 features to predict house prices. It will set the coefficients of irrelevant features (like house color) to zero.	If we have 100 features it will reduce the impact of every feature but won't completely remove any.	If we have features like "size" and "rooms" that are similar it will remove one and shrink the other.

Optimization

- Optimization techniques in **deep learning** refer to the **algorithms** and **methods** used to **adjust** the **parameters** of a **neural network** during **training** to **minimize** a **loss function** and **improve model performance**.
- These **techniques** aim to **find** the **optimal set** of **weights** and **biases** that **allow** the **model** to make **accurate predictions**.

Hyperparameter Tuning

- The process of finding the best values for **hyperparameters** — the settings you choose *before* training a model (e.g., λ in Ridge/Lasso, learning rate, number of hidden layers).

Key point:

- Hyperparameters are **not learned** from the data; we have to **manually search** for them using methods like:
 - Grid Search
 - Random Search
 - Bayesian Optimization
 - Cross-Validation-based tuning

Optimization

- The mathematical process the model uses **during training** to find the **best parameters** (also called coefficients or weights, β in regression) that minimize a **loss function**.

Key point:

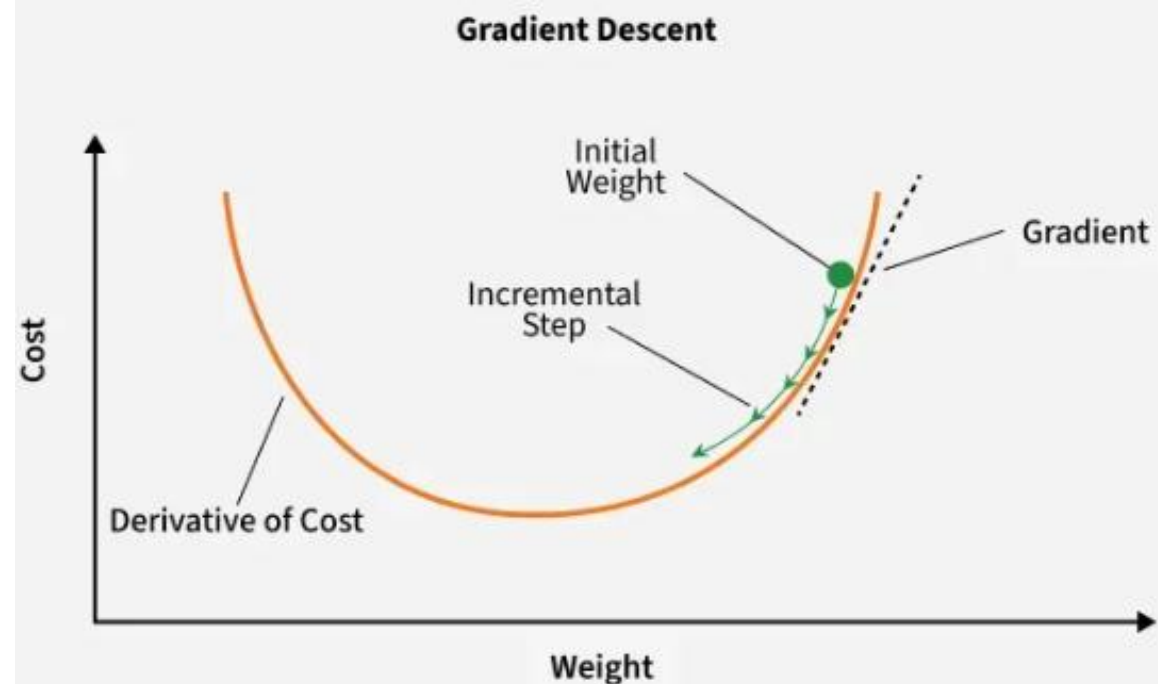
- Optimization happens **inside the training loop** and uses algorithms like:
 - Gradient Descent
 - Stochastic Gradient Descent (SGD)
 - Adam, RMSprop, etc.

Optimization

- In ML & DL, **optimizers** and **loss functions** are **two fundamental components** that help **improve** a **model's performance**.
 - A **loss function** evaluates a **model's effectiveness** by **computing** the **difference** between **expected** and **actual outputs**. Common loss functions include **log loss**, **hinge loss** and **mean square loss**.
 - An **optimizer** improves the **model** by **adjusting** its **parameters** (**weights** and **biases**) to **minimize** the **loss function value**.
 - **Examples** include **RMSProp**, **ADAM** and **SGD** (Stochastic Gradient Descent).
- The **optimizer's** role is to **find** the **best combination** of **weights** and **biases** that **leads** to **the most accurate predictions**.

Gradient Descent

- It is a **popular** optimization **method** for **training** ML& DL models.
- It **works** by **iteratively adjusting** the **model parameters** in the **direction** that **minimizes** the **loss function**.



Key Steps in Gradient Descent

- 1.Initialize parameters:** Randomly initialize the model parameters.
- 2.Compute the gradient:** Calculate the gradient (derivative) of the loss function with respect to the parameters.
- 3.Update parameters:** Adjust the parameters by moving in the opposite direction of the gradient, scaled by the learning rate.

• **Formula :**
$$\theta_{(k+1)} = \theta_k - \alpha \nabla J(\theta_k)$$

Where,

- $\theta_{(k+1)}$ is the *updated parameter vector* at the $(k+1)^{th}$ iteration
- θ_k is the *current parameter* vector at the k^{th} iteration
- α is the *learning rate*, which is a *positive scalar* that determines the *step size* for *each iteration*
- $\nabla J(\theta_k)$ is the *gradient* of the *cost* or *loss function J* with *respect* to the *parameters* θ_k

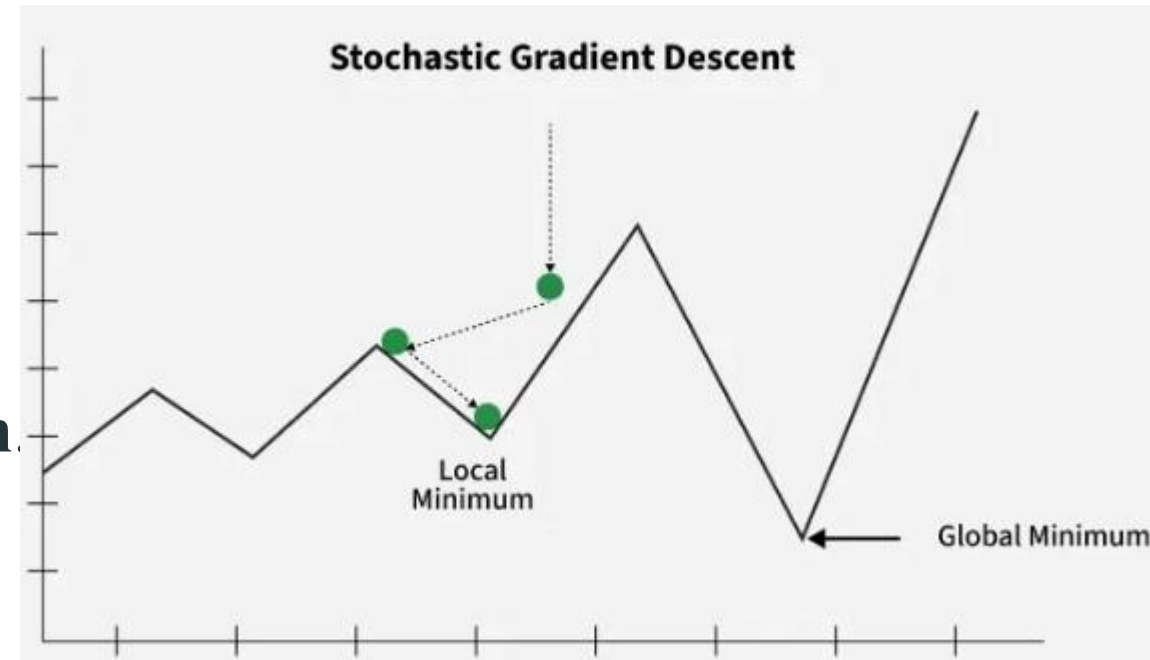
Variants of Gradient Descent

1. Stochastic Gradient Descent (SGD)

- **SGD** updates the model parameters after each training example, making it more efficient for large datasets compared to traditional **GD**, which uses the entire dataset for each update.

- **Steps:**

1. **Select** a **training example**.
2. **Compute** the **gradient** of the **loss function**.
3. **Update** the model **parameters**.



Advantages: Requires less memory and may find new minima.

Disadvantages: Noisier, requiring more iterations to converge.

2. Mini Batch Gradient Descent

- **It** consists of a **predetermined number of training examples, smaller than the full dataset**. This approach **combines the advantages of the previously mentioned variants**.

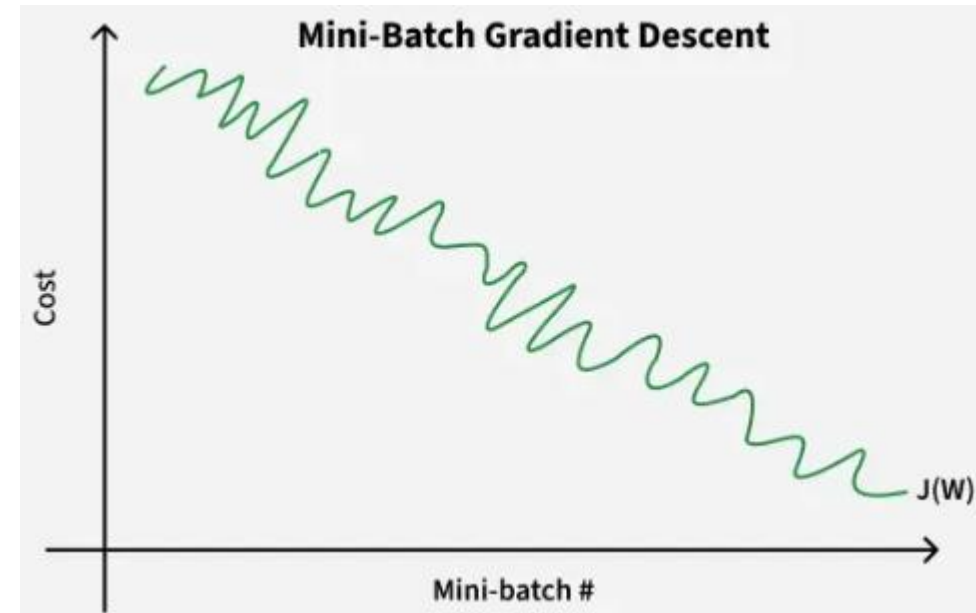
In **one epoch**, following the **creation of fixed-size mini-batches**, we **execute the following steps**:

1. **Select a mini-batch.**

2. **Compute the mean gradient of the mini-batch.**

3. **Apply the mean gradient obtained in step 2 to update the model's weights.**

4. **Repeat steps 1 to 2 for all the mini-batches that have been created.**



- **Advantages:** Requires a **medium amount of memory** and **less time is required to converge** when compared to SGD
- **Disadvantage:** May get **stuck at local minima**

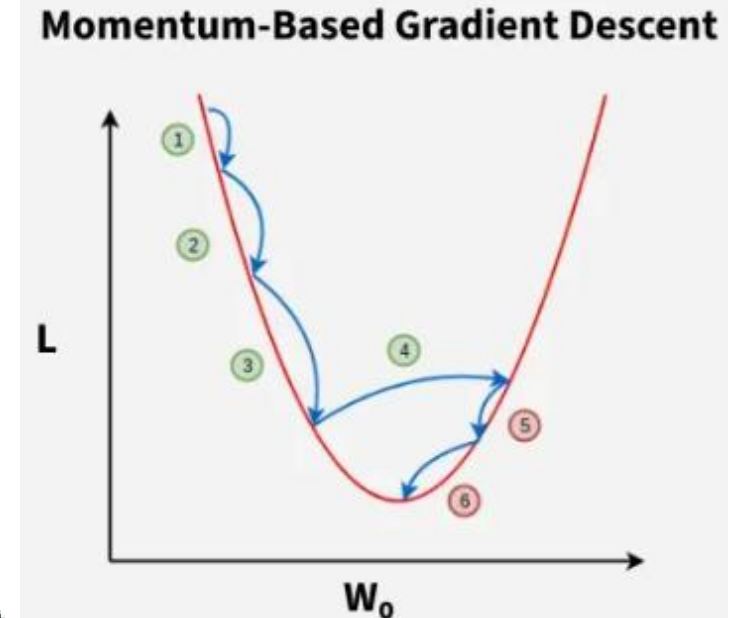
3. SGD with Momentum

Momentum helps accelerate convergence by smoothing out the noisy gradients of SGD, thus reducing fluctuations and improving the speed of convergence.

$$v_{(t+1)} = \beta * v_t + (1 - \beta) * \nabla J(\theta_t)$$

Where

- v_t is the **momentum** at time t .
 - β is the **momentum coefficient**.
 - $\nabla J(\theta_t)$ is the **gradient** at **time** t .
- Then, the **model parameters** are **updated** using: $\theta_{(t+1)} = \theta_t - \alpha * v_{(t+1)}$
 - **Advantages:** Mitigates oscillations, reduces variance and faster convergence.
 - **Disadvantages:** Requires tuning the momentum coefficient β .



Advanced Optimizers

1. AdaGrad (Adaptive Gradient Algorithm)

- **It** adapts the learning rate for each parameter based on the **historical gradient information**. The learning rate decreases over time, making AdaGrad effective for **sparse features**.

$$\theta_{(t+1)} = \theta_t - \frac{\alpha}{\sqrt{G_t + \epsilon}} * \nabla J(\theta_t)$$

- *Where: G_t is the sum of squared gradients.*
 - ϵ is a *small constant to avoid division by zero*.
- **Advantages:** Adapts the learning rate, improving training efficiency.
- **Disadvantages:** Learning rate decays too quickly, causing slow convergence.

2. RMSProp (Root Mean Square Propagation)

- **RMSProp** improves upon AdaGrad by introducing a decay factor to prevent the learning rate from decreasing too rapidly.

$$E[g^2]_t = \gamma * E[g^2]_{(t-1)} + (1 - \gamma) * (\nabla J(\theta_t))^2$$
$$\theta_{(t+1)} = \theta_t - \frac{\alpha}{\sqrt{E[g^2]_t + \epsilon}} * \nabla J(\theta_t)$$

- *Where:*
 - γ is the decay rate.
 - $E[g^2]_t$ is the exponentially moving average of squared gradients.
- **Advantages:** Prevents excessive decay of learning rates.
- **Disadvantages:** Computationally expensive due to the additional parameter.

• 3. Adam (Adaptive Moment Estimation)

- **Adam** combines the advantages of **Momentum** and **RMSProp**. It uses both the first moment (mean) and second moment (variance) of gradients to adapt the learning rate for each parameter.

1. Update the first moment: $m_t = \beta_1 * m_{(t-1)} + (1 - \beta_1) * \nabla J(\theta_t)$

2. Update the second moment: $v_t = \beta_2 * v_{(t-1)} + (1 - \beta_2) * (\nabla J(\theta_t))^2$

3. Bias correction: $\hat{m}_t = \frac{m_t}{1 - \beta_1^t}$, $\hat{v}_t = \frac{v_t}{1 - \beta_2^t}$

4. Update parameters: $\theta_{(t+1)} = \theta_t - \frac{\alpha}{\sqrt{\hat{v}_t + \epsilon}} * \hat{m}_t$

Where:

- β_1 and β_2 are the decay rates for the first and second moments.
- ϵ is a small constant to prevent division by zero.
- **Advantages:** Fast convergence.
- **Disadvantages:** Requires significant memory due to the need to store first and second moment estimates

Comparison of Optimizers

Optimizer	Advantages	Disadvantages
SGD	Simple, easy to implement	Slow convergence, requires tuning
Mini-Batch SGD	Faster than SGD	Computationally expensive, stuck in local minima
SGD with Momentum	Faster convergence, reduces noise	Requires careful tuning of β
AdaGrad	Adaptive learning rates	Decays too fast, slow convergence
RMSProp	Prevents fast decay of learning rates	Computationally expensive
Adam	Fast, combines momentum and RMSProp	Memory-intensive, computationally expensive

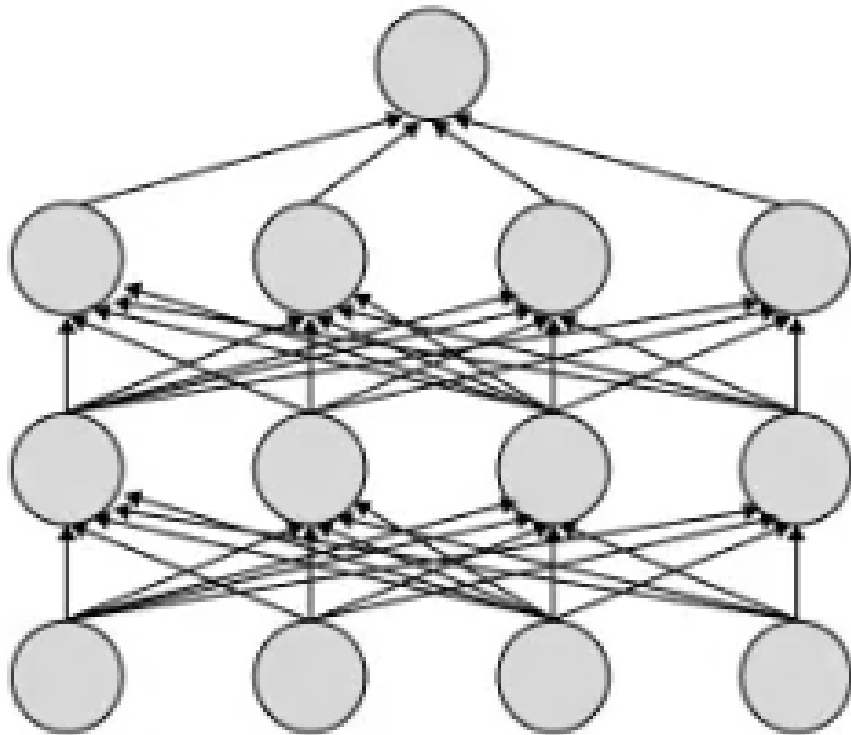
Dropout

- **Training deep learning** models for **too long** on the **same data** can lead to **overfitting** where the **model performs well** on **training data** but **poorly** on **unseen data**.
- Dropout Regularization **helps overcome** this by **randomly deactivating** a **portion** of **neurons** during **training** hence forcing the **model** to **learn more robust** and **independent features**.

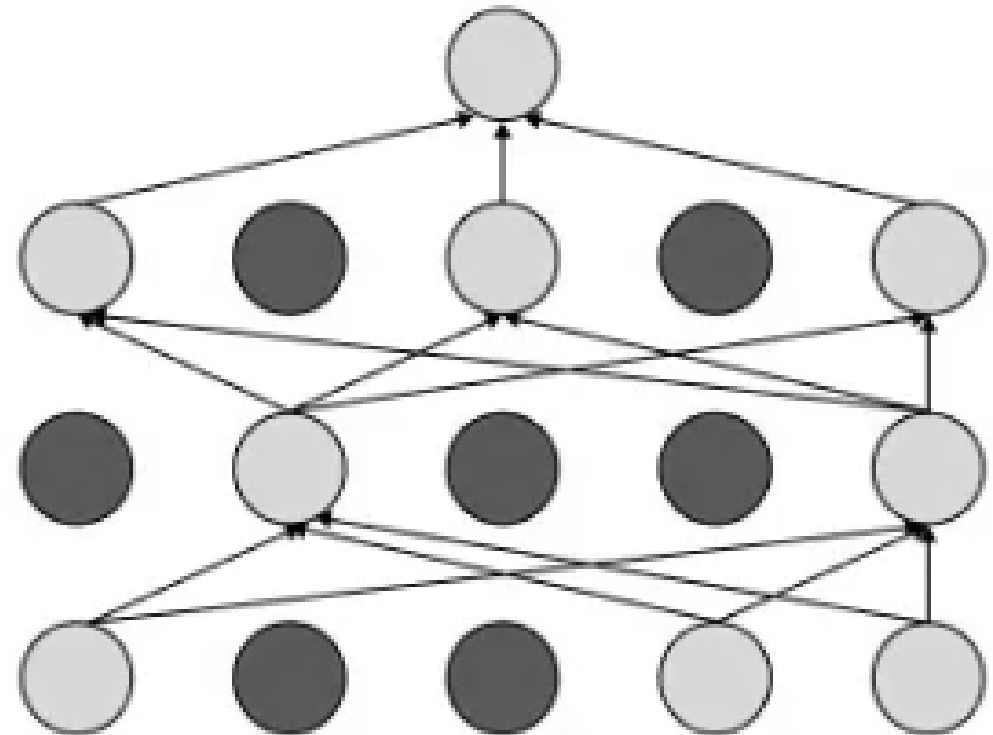
Dropout

- This image shows how **dropout works** by **randomly deactivating neurons during training**, forcing the **network** to **rely** on **multiple paths** and **preventing overfitting**

A neural network before dropout

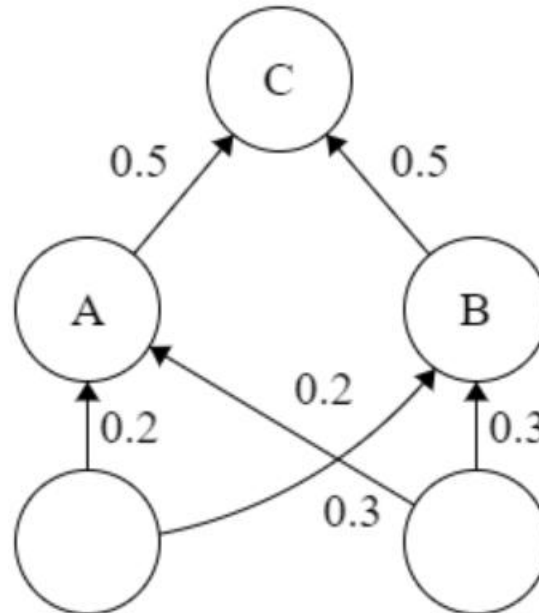


A neural network after dropout



➤ Why Dropout is needed

- When a **fully-connected layer** has a **large number of neurons**, **co-adaptation** is **more likely to happen**.
- **Co-adaptation** refers to when **multiple neurons in a layer extract the same, or very similar, hidden features** from the **input data**.
- This can **happen** when the **connection weights** for **two different neurons** are **nearly identical**.



This **poses two different problems to the model:**

- **Wastage of machine's resources when computing the same output.**
- If **many neurons** are **extracting** the **same features**, it **adds more significance** to **those features** for the model.
- This **leads** to **overfitting** if the **duplicate extracted features** are **specific** to **only** the **training** set.

Solution-How it Works

- Dropout works by **randomly turning off** a **fraction** of **neurons** during **each training** pass. **When** a **neuron** is **dropped**, it **sends no output** and receives **no weight updates** for that **batch**.
- The **dropout rate** " P " determines the **probability** of removing a neuron.
- A neuron output h is modified as

$$h' = h \cdot m$$

Where m : Dropout mask randomly sampled for each neuron.

$m=1$: neuron is kept,

*$m=0$: neuron is **dropped***

Solution-How it Works

- To keep **activation values consistent during training**, **inverted dropout** is **used** where **active neurons** are **scaled** as:

$$h' = \frac{h \cdot m}{1 - P}$$

Where ***h***: **Original output** of a **neuron before dropout**.

h': Output **after** applying **dropout** (**scaled** or **zeroed out**).

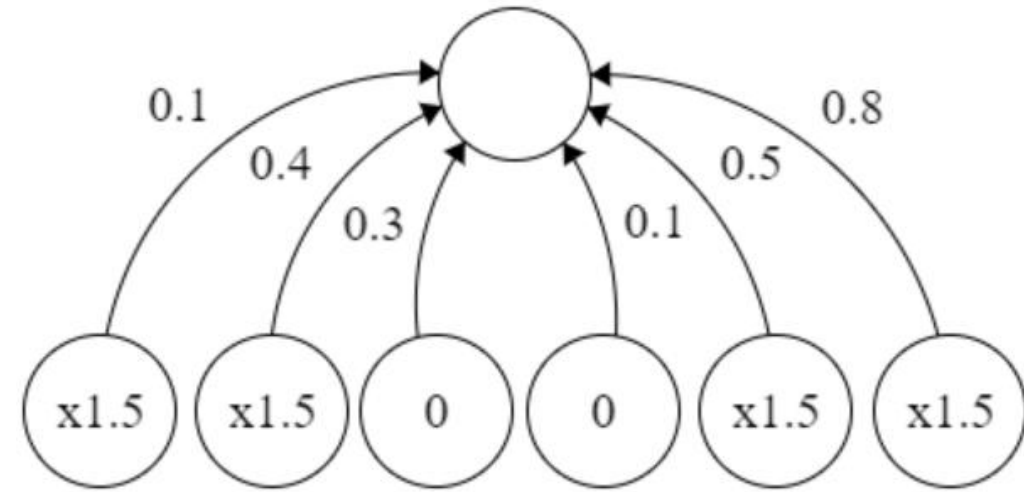
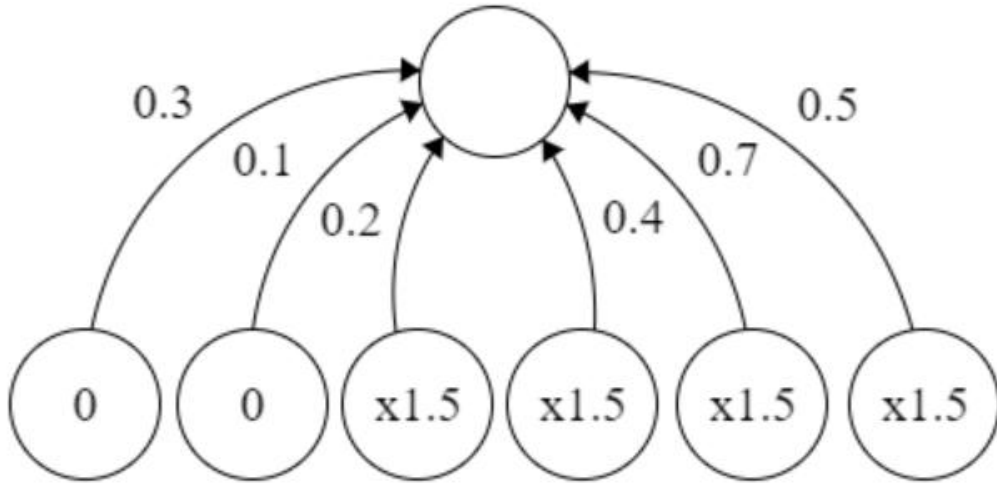
- This ensures that the expected output remains stable even when some neurons are dropped.

Note: **During testing, dropout is disabled**, meaning **all neurons remain active** and **no randomness is applied**. The **network** uses its **full capacity**, and **activations** are **scaled appropriately** to **match the expected values** from **training**.

Why dropout works?

- By using dropout, **in every iteration**, you **will work** on a **smaller neural network than** the **previous one** and **therefore**, it **approaches regularization**.
- Dropout **helps in shrinking** the **squared norm** of the **weights** and this **tends to a reduction in overfitting**.

Drop-out



- The **two images** represent **dropout applied** to a **layer of 6 units**, shown at **multiple training steps**.
- The **dropout rate** is **1/3**, and the **remaining 4 neurons** at **each training step** **have** their **value scaled** by **x1.5**.
- Thereby, we are **choosing a random sample of neurons rather than training the whole network at once**.
- This **ensures** that the **co-adaptation** is **solved** and they **learn** the **hidden features better**.

The key features of dropout:

- Randomly deactivates a set percentage of neurons in each training step.
- Works with dense, convolutional and recurrent layers.
- Dropout rate controls how many neurons are dropped.
- Remaining active neurons are scaled to maintain output stability.
- Reduces co-dependency among neurons and improves generalization.

Types of Dropout Regularization

- **Standard Dropout:** Randomly removes individual neurons during training to reduce overfitting.
- **Spatial Dropout:** Drops entire feature maps in CNNs to preserve spatial structure.
- **Dropout2D / Dropout3D:** Applies dropout across 2D or 3D feature channels for images and volumetric data.
- **DropConnect:** Randomly drops weights instead of neurons, offering stronger regularization.
- **Alpha Dropout:** Maintains activation statistics and works best with SELU-based networks.
- **Variational Dropout:** Uses the same dropout mask across time steps in RNNs for stable learning.
- **Monte Carlo Dropout:** Keeps dropout active at inference to estimate prediction uncertainty.

Batch Normalization

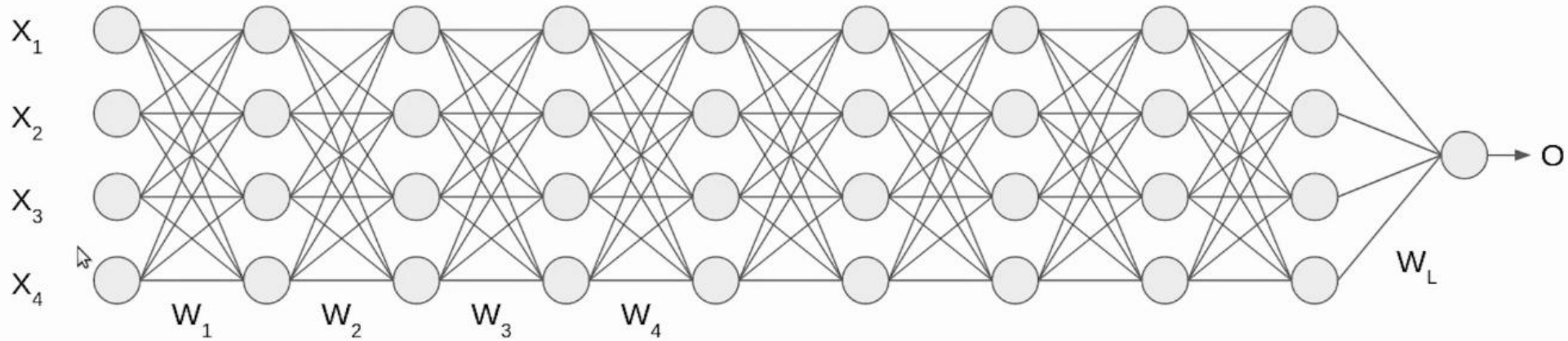
- **Batch Normalization in Deep Learning** is a **technique** used to **normalize** the **activations** of a **neural network** during **training**.
 - It **reduces** the **problem** of **internal covariate shift**, where the **distribution** of **inputs** to **each layer** **changes** as the **model learns**.
 - By **normalizing** the **activations** of **each mini-batch**, **Batch Normalization** **keeps** the **input values** of **each layer** **more consistent**, **resulting** in **faster** and **more stable training**.

Batch Normalization

Need for Batch Normalization

- **Speeds Up Training:** Stable activations **allow** the **network** to **converge** in **fewer** training iterations.
- **Supports Higher Learning Rates:** Makes **training** more **robust**, **allowing larger learning rates without causing divergence**.
- **Improves Gradient Flow:** **Reduces** the **chances** of **vanishing** and **exploding** gradients in deep neural networks.
- **Acts as a Regularizer:** The **variation** introduced by **mini-batch** statistics can **reduce overfitting**, **sometimes decreasing** the **need** for **dropout**.
- **Reduces Sensitivity to Weight Initialization:** Models **become less dependent** on carefully **chosen** initial **parameter** values, making **training more reliable**.

- For **example**, We have a deep neural network, as shown in the following image.

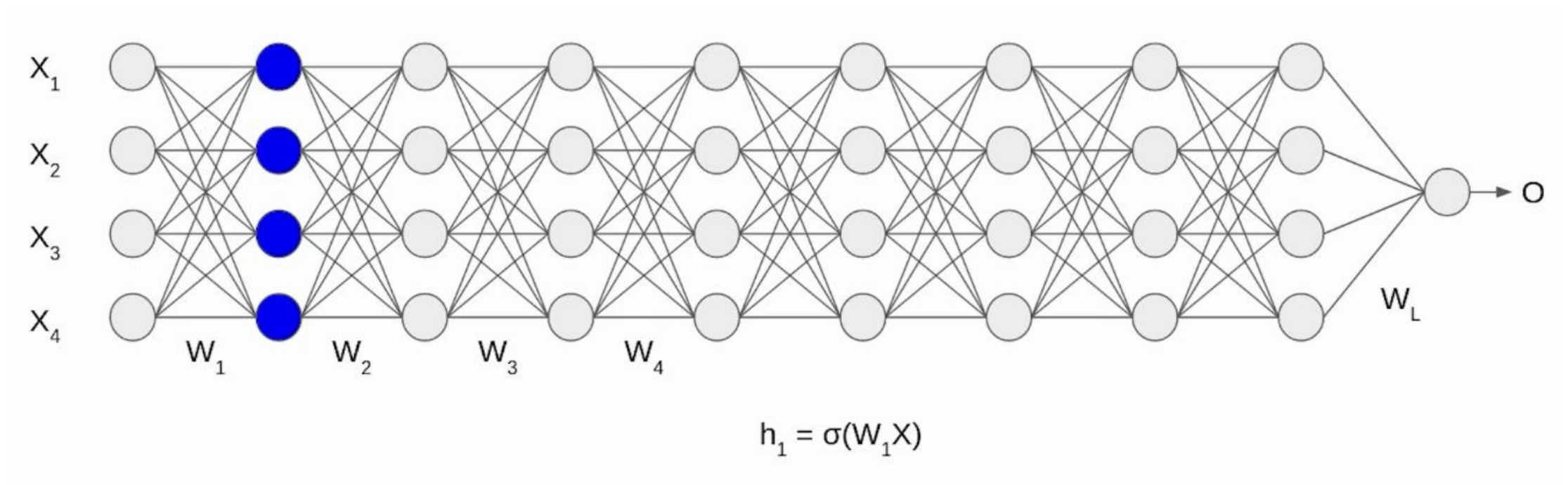


L = Number of layers

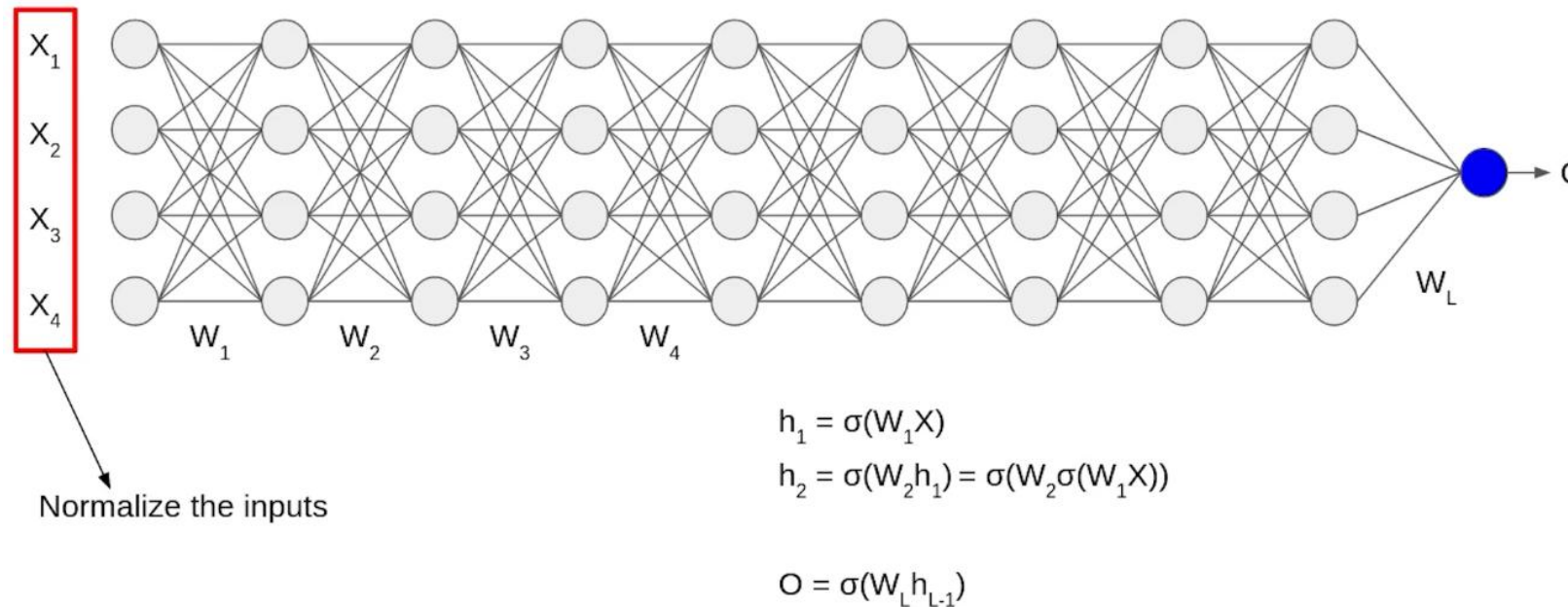
Bias = 0

Activation Function: Sigmoid





- Initially, our inputs X_1 , X_2 , X_3 , and X_4 are normalized as they come from the pre-processing stage.
- When the input passes through the first layer, it transforms as a **sigmoid function** applied over the dot product of input X and the weight matrix W .
- Similarly, this transformation will take place for the second layer and continue until the last layer L , as shown in the above image.



- Although our input X was normalized with time, the output will no longer be on the same scale. As the data pass through multiple layers of the neural network and L activation functions are applied, it leads to an internal co-variate shift in the data.

How does Batch Normalization work?

- It is a two-step process. **First**, the **input** is **normalized**, and **later**, **rescaling** and **offsetting** are performed.

Normalization of the Input

- Normalization is the **process** of **transforming** the **data** to **have** a **mean zero** and **variance one**.

➤ Working

✓ Step 1: Compute the Batch Mean

- For **each mini-batch**, the **mean** of the **activations** is **calculated**. The batch mean represents the **average value** of **all activations** in the **current mini-batch** and is used as the reference for normalization.

$$\mu_B = \frac{1}{m} \sum_{i=1}^m x_i$$

Where, m is the number of samples in the mini-batch.

x_i is the activation of the i th sample, μ_B is the mean of the mini-batch.

✓ Step 2: Compute the Batch Variance

- **Next**, the **variance** of the **activations** is **calculated**. The variance measures **how far** the **activations** are **spread around** the **batch mean**.

$$\sigma_B^2 = \frac{1}{m} \sum_{i=1}^m (x_i - \mu_B)^2$$

Where, σ_B^2 is the variance of the mini-batch.

✓ Step 3: Normalize the Activations

- Each activation is normalized by subtracting the batch mean and dividing by the square root of the batch variance. A small constant ϵ is added to the denominator to prevent division by zero.

$$x_i = \frac{x_i - \mu_B}{\sqrt{\sigma_B^2 + \epsilon}}$$

Where, x_i is the normalized activation
 ϵ is a small constant added for numerical stability

After this step, the activations have approximately zero mean and unit variance, which helps stabilize the learning process.

✓ Step 4: Scale and Shift the Normalized Activations

- Normalizing the activations may reduce the flexibility of the network. To overcome this, Batch Normalization introduces two learnable parameters:
- The final output is computed as:

$$y_i = \gamma \hat{x}_i + \beta$$

Where, y_i is the final output after Batch Normalization
 γ controls the scale of the normalized activations
 β shifts the normalized activations to the desired range

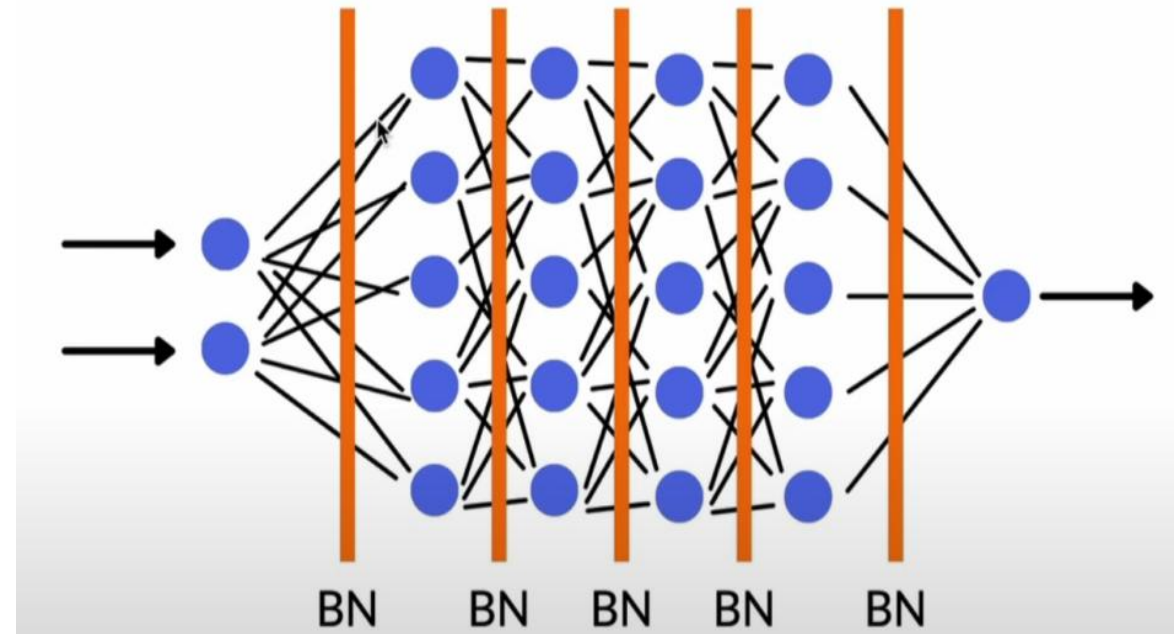
✓ Step 5: Pass the Output to the Next Layer

- The scaled and shifted activations are then passed to the next layer of the neural network.
- During training, the normalization statistics (mean and variance) are computed separately for each mini-batch.
- During inference, the layer uses the running averages of these statistics collected during training to ensure consistent predictions.

Without γ and β , BN would always force outputs to zero mean and unit variance which can limit the network's expressiveness.

With γ and β :

- The network decides how much normalization to keep.
- If BN is *not* beneficial for some layer, the network can learn $\gamma = \sigma$ and $\beta = \mu$, effectively reverting BN.



Example

1. A mini-batch consists of the following outputs from a hidden layer:

- $X=[8,10,12,14]$,
- Assume: Scale parameter $(\gamma) = 2$,
- Shift parameter $(\beta) = 1, \epsilon \approx 0$ (ignore for simpliity), $\mu=11$

Perform **Batch Normalization** by calculating: Mean, Variance, Normalized values, Final output after scaling and shifting

Step 1: Calculate the Mean

$$\begin{aligned}\mu &= \frac{8 + 10 + 12 + 14}{4} \\ &= \frac{44}{4}\end{aligned}$$

Step 2: Calculate the Variance

$$\sigma^2 = \frac{(8 - 11)^2 + (10 - 11)^2 + (12 - 11)^2 + (14 - 11)^2}{4}$$

$$= \frac{(-3)^2 + (-1)^2 + 1^2 + 3^2}{4}$$

$$= \frac{9 + 1 + 1 + 9}{4}$$

$$\sigma^2=5$$

Step 3: Calculate the Standard Deviation

$$\sigma = \sqrt{5}$$

$$\sigma = 2.236$$

Step 4: Normalize Each Value

$$\hat{x} = \frac{x - \mu}{\sigma}$$

For 8

$$\hat{x} = \frac{8 - 11}{2.236} = \frac{-3}{2.236} = -1.34$$

For 10

$$\hat{x} = \frac{10 - 11}{2.236} = -0.45$$

For 12

$$\hat{x} = \frac{12 - 11}{2.236} = 0.45$$

For

$$\hat{x} = \frac{14 - 11}{2.236} = 1.34$$

Original Value	Normalized Value
8	-1.34
10	-0.45
12	0.45
14	1.34

Step 5: Apply Scaling and Shifting

- Batch Normalization does not stop after normalization. It applies $y = \gamma \hat{x} + \beta$

- Where $\gamma=2, \beta=1$

- For 8

$$\begin{aligned} y &= 2(-1.34) + 1 \\ &= -2.68 + 1 \\ &= -1.68 \end{aligned}$$

- For 10

$$\begin{aligned} y &= 2(-0.45) + 1 \\ &= -0.90 + 1 \\ &= 0.10 \end{aligned}$$

- For 12

$$\begin{aligned} y &= 2(0.45) + 1 \\ &= 0.90 + 1 \\ &= 1.90 \end{aligned}$$

- For 14

$$\begin{aligned} y &= 2(1.34) + 1 \\ &= 2.68 + 1 \\ &= 3.68 \end{aligned}$$

Input	Normalized ($\hat{x}$)	Final Output $y = \gamma \hat{x} + \beta$
8	-1.34	-1.68
10	-0.45	0.10
12	0.45	1.90
14	1.34	3.68

- The learnable parameters γ (**gamma**) and β (**beta**) allow the network to adjust the normalized values to the scale and position that best suits the learning process.

Data Augmentation

- Data augmentation is a technique of artificially increasing the training set by creating modified copies of a dataset using existing data.
- It includes making minor changes to the dataset or using deep learning to generate new data points.

➤ Augmented vs. synthetic data

- Data augmentation and synthetic data generation are distinct yet complementary techniques in machine learning:

Augmented data: This involves creating modified versions of existing data to increase dataset diversity. For example, in image processing, applying transformations like rotations, flips, or color adjustments to existing images can help models generalize better.

Synthetic data: This refers to artificially generated data, which allows researchers and developers to test and improve algorithms without risking the privacy or security of real-world data.

- Data augmentation helps machine learning models perform better by making the most of existing data.
- It prevents overfitting, improves accuracy, and creates diversity in training data, which is crucial when datasets are small or imbalanced.
- By simulating real-world variations, it makes models more robust and reliable without the need for expensive data collection. In short, it's a simple, powerful way to help models learn and generalize effectively.

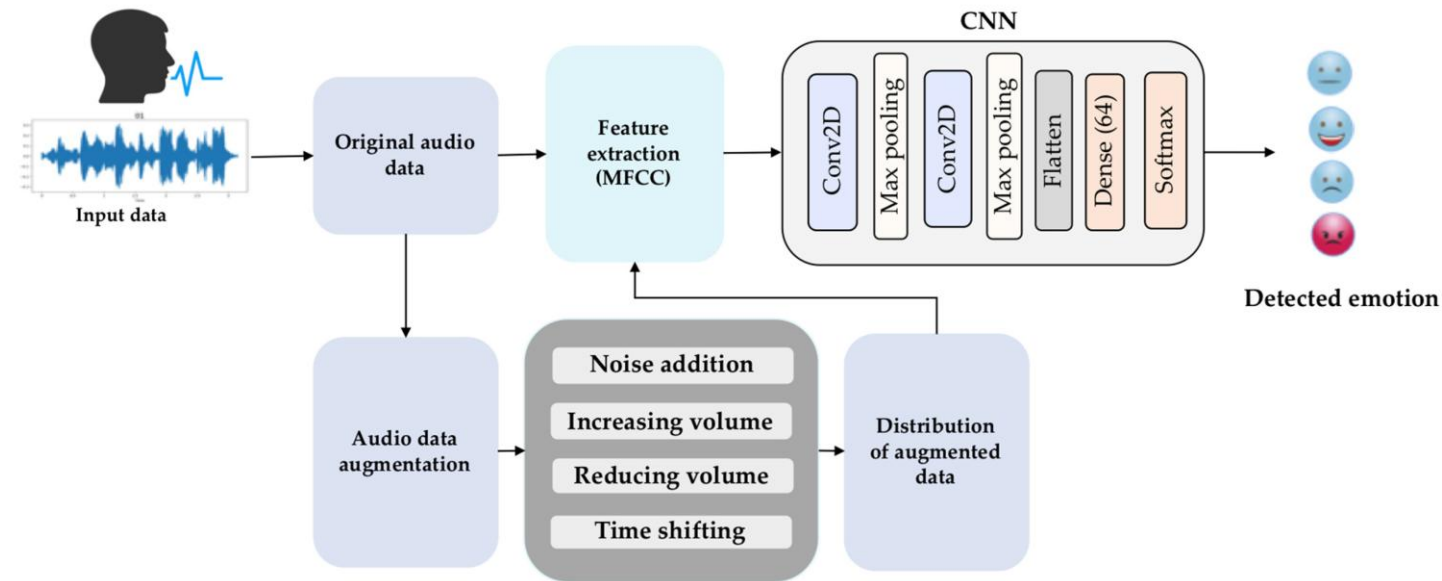
Limitations of data augmentation

- The biases in the original dataset persist in the augmented data.
- Quality assurance for data augmentation is expensive.
- Research and development are required to build a system with advanced applications. For example, generating high-resolution images using GANs can be challenging.
- Finding an effective data augmentation approach can be challenging.

Data Augmentation Techniques

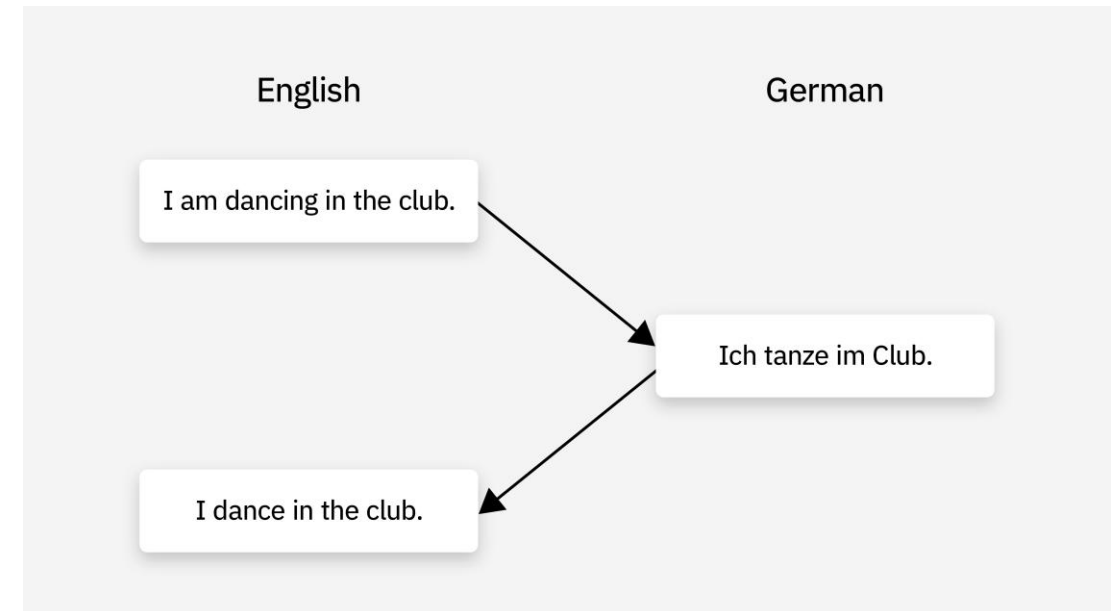
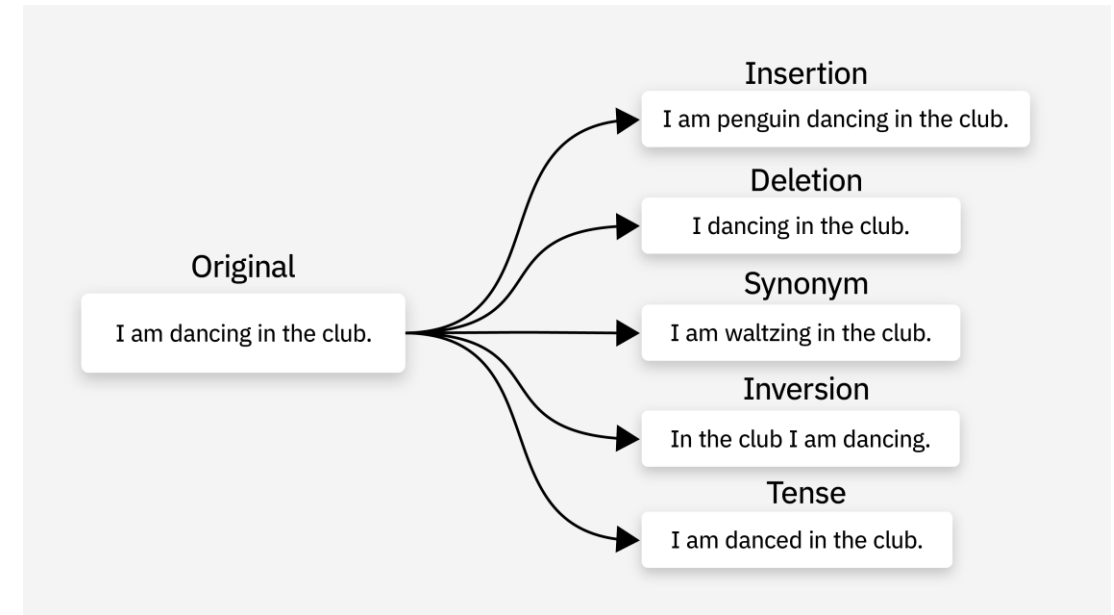
1. Audio Data Augmentation

- **Noise injection:** add gaussian or random noise to the audio dataset to improve the model performance.
- **Shifting:** shift audio left (fast forward) or right with random seconds.
- **Changing the speed:** stretches times series by a fixed rate.
- **Changing the pitch:** randomly change the pitch of the audio.



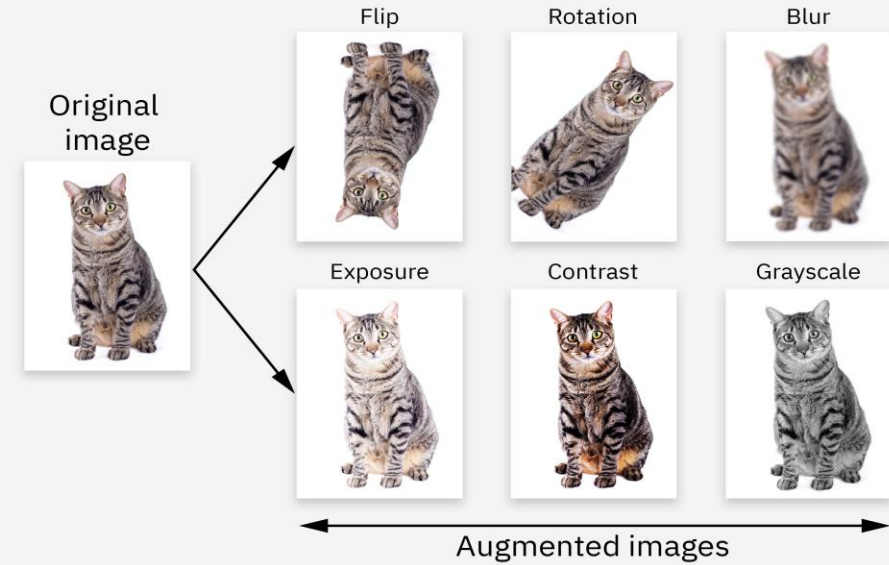
2. Text Data Augmentation

- **Word or sentence shuffling:** randomly changing the position of a word or sentence.
- **Word replacement:** replace words with synonyms.
- **Syntax-tree manipulation:** paraphrase the sentence using the same word.
- **Random word insertion:** inserts words at random.
- **Random word deletion:** deletes words at random.



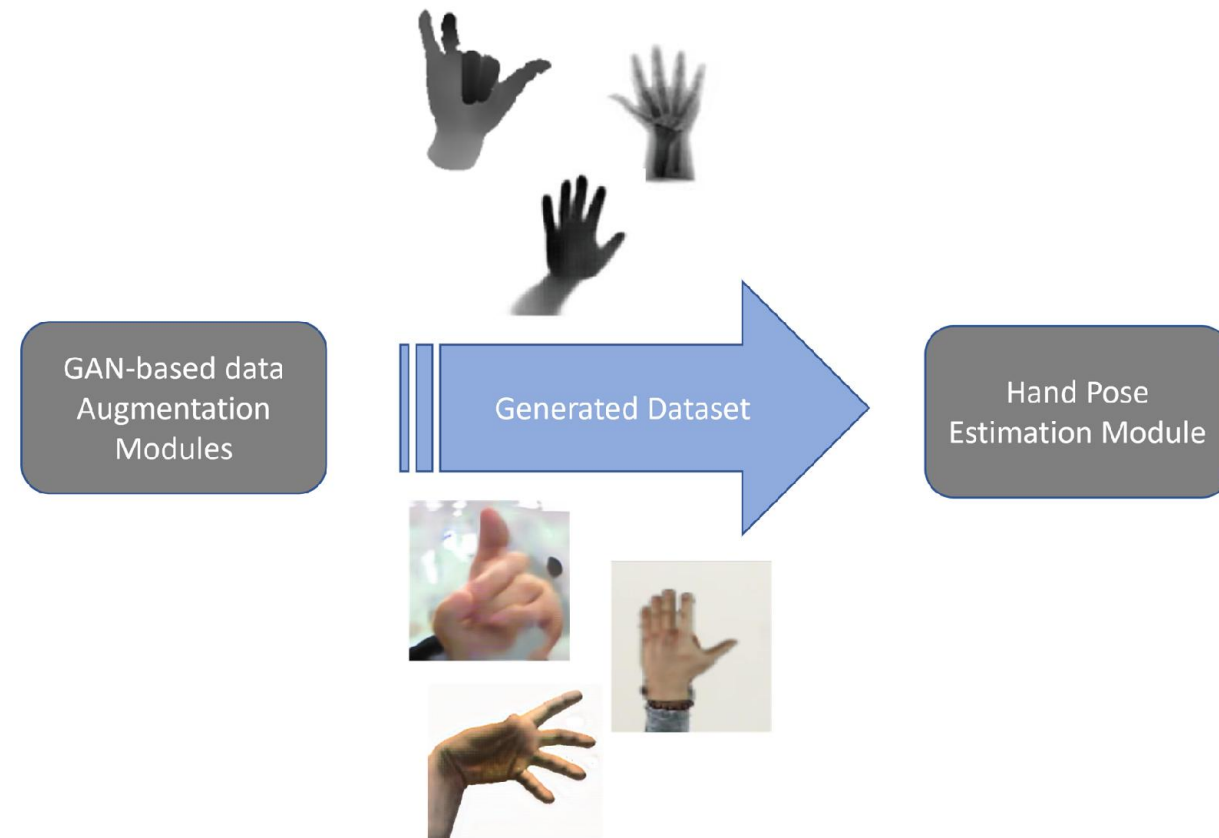
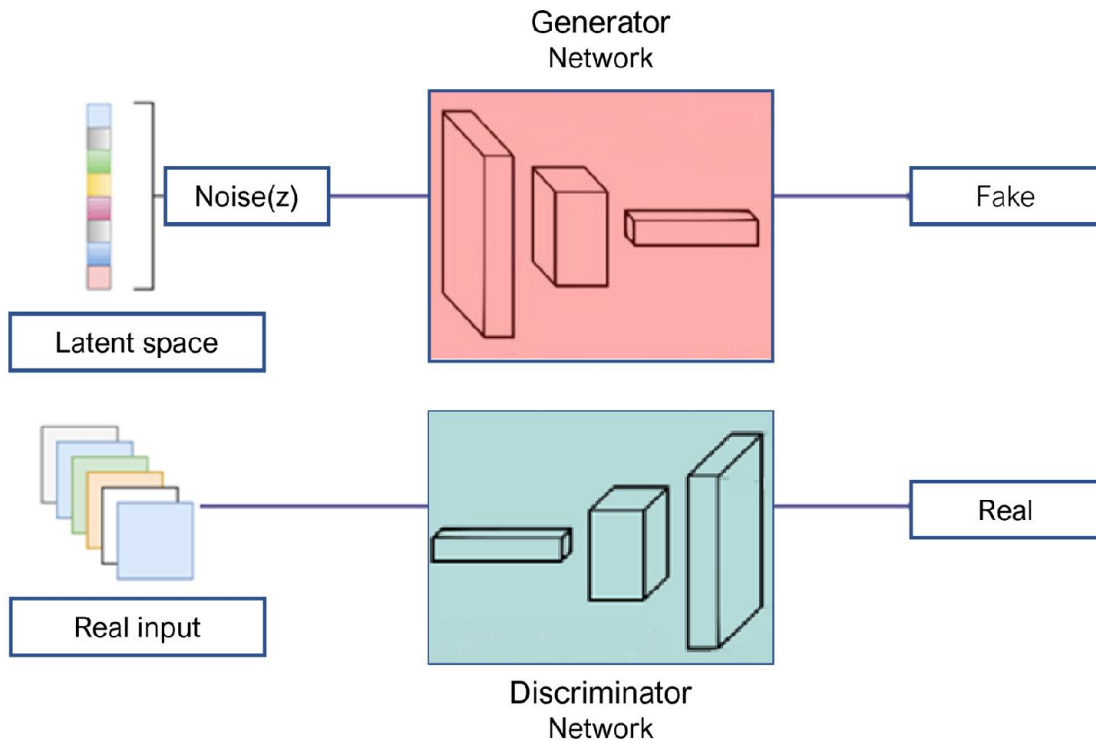
3. Image Augmentation

- **Geometric transformations:** randomly flip, crop, rotate, stretch, and zoom images. You need to be careful about applying multiple transformations on the same images, as this can reduce model performance.
- **Color space transformations:** randomly change RGB color channels, contrast, and brightness.
- **Kernel filters:** randomly change the sharpness or blurring of the image.
- **Random erasing:** delete some part of the initial image.
- **Mixing images:** blending and mixing multiple images.



4.Synthetic Data Generation

- **Generative adversarial networks (GANs):** used to generate new data points or images. It does not require existing data to generate synthetic data.
- **Neural Style Transfer:** a series of convolutional layers trained to deconstruct images and separate context and style.



End of Module-II